Federated O-RAN Slice SLA Prediction: A Cross-Architecture Empirical Benchmark on Colosseum/ColO-RAN

Hao-Chun Tsai, *Student Member, IEEE*

***Abstract*—Federated learning (FL) on O-RAN edge gNBs is the natural paradigm for slice-level SLA-violation forecasting because per-user telemetry cannot be centrally pooled. Despite a decade of FL benchmark research, the deployment design space — sequence model architecture, federation algorithm, client-heterogeneity partition, commodity edge hardware — has been studied one axis at a time, leaving the joint behaviour under realistic O-RAN traffic characterised only anecdotally. This paper presents the first cross-architecture empirical FL benchmark on the publicly-available Colosseum/ColO-RAN testbed: 3 sequence backbones (LSTM, Mamba, Spiking-SSM) $\times$ 5 algorithms (FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn) $\times$ 6 partitions (natural-by-BS $+$ Dirichlet $\alpha \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$) $\times$ 10 seeds $=$ 900 cells, with NVML idle-baseline-subtracted training-energy measurement on RTX 4080 commodity hardware, augmented by a 15-cell controlled random-split ablation on $4\times$ Tesla V100-SXM2-32GB to probe the leading mechanism hypothesis. We report four empirical findings: (1) the natural base-station partition uniformly outperforms every parametric Dirichlet α across all 3 architectures $\times$ 5 algorithms in our setup — preserving the prediction-relevant structural coherence of one-gNB-per-client that artificial Dirichlet partitioning destroys, inverting the standard FL-benchmark assumption that lower α is always harder; (2) the algorithm-design space is flat at FedAdam-saturated headroom ($\leq +0.016$ AUC over FedAvg), bounding the room available to recent sharpness-aware methods; (3) selective-state-space backbones interact catastrophically with control-variate algorithms at moderate partition skew (Mamba $\times$ SCAFFOLD $\times$ $\alpha \in \{0.10, 0.50\}$); (4) architecture choice traverses $10\times$ in commodity-GPU energy and an order-of-magnitude larger AUC range than algorithm choice, dwarfing the algorithmic-leverage scale. We provide a reference benchmark with paired-bootstrap CI$_{95}$ over 270 paired distributions, NVML-instrumented per-cell energy, and pre-registered statistical protocol; the artefact is released under Apache-2.0 with Croissant dataset metadata and a Zenodo DOI on paper acceptance.**

***Index Terms*—Federated learning, O-RAN, slice management, SLA prediction, state-space models, spiking neural networks, NVML energy measurement, paired-bootstrap inference.**

I. Introduction

OPEN Radio Access Networks (O-RAN) decomposes the cellular base station into commodity hardware components controlled by ML-driven xApps and rApps running on disaggregated RAN Intelligent Controllers (RICs) [1]. Slice-level Service Level Agreement (SLA) violation forecasting is one near-RT RIC xApp pattern of practical interest: each gNB simultaneously serves heterogeneous traffic slices (the ColO-RAN release exercises eMBB constant-bitrate, MTC Poisson, and URLLC Poisson — see Section III), and proactive forecasting of next-second SLA risk informs near-RT RIC resource-allocation decisions within its 10 ms – 1 s control loop. Because edge gNBs accumulate per-user telemetry that cannot be centrally pooled, federated learning (FL) is the natural training paradigm: each gNB trains a local sequence model on its own traces and shares only weight updates with the FL aggregator. We place the aggregator in the non-RT RIC / SMO orchestration layer ($\geq$ 1 s timescale fits our 100-round federated training cadence per Section IV); a near-RT-RIC-internal xApp aggregator would alternatively be possible if cross-cell xApp messaging supports the federation protocol — we do not engage with that deployment alternative beyond noting it.

Standard FL practice — codified by a decade of toy-image benchmarks — treats client heterogeneity as a *wound* to be healed: lower Dirichlet α (more skew) drives lower accuracy, motivating sharpness-aware methods, control variates, and proximal regularizers. **In O-RAN slice SLA prediction, this premise inverts.** When clients are the *natural* base stations of a Colosseum/ColO-RAN testbed (one client per gNB), the federated average across 7 base-station-natural clients consistently outperforms every uniform-Dirichlet partition of the same training data on the same model + algorithm. Empirically (Section VI, all values are out-of-sample test AUC averaged over 10 seeds), the natural-by-BS partition achieves AUC 0.913–0.918 on LSTM (5-algo range; $\Delta \approx 0.005$), 0.8998–0.9186 on Mamba (with SCAFFOLD as the lower outlier and the other four algorithms in 0.911–0.919), and 0.840–0.856 on Spiking-SSM, **strictly above the AUC obtained at any parametric Dirichlet** α for every (arch, algo) pair, and monotonically increasing as $\alpha \to 0$ within the Dirichlet sweep. The $\alpha = 5.0 \to \alpha = 0.05$ AUC gap is $\approx$ 0.10–0.12 on the dense LSTM/Mamba backbones; on Spiking-SSM the gap is smaller (≈ 0.022) but still positive, and across all five algorithms heterogeneity is associated with higher AUC, not lower. The mechanism is open — natural-by-BS is *not* the low-α limit of the Dirichlet sweep (the two partition families redistribute rows along orthogonal axes, see Section IV-C) — and we present a controlled `random_split` ablation in Section VII-A1 (15 cells $\times$ 3 architectures on $4\times$ Tesla V100-SXM2-32GB) showing that any partition that breaks bs grouping (whether by per-slice Dirichlet redistribution at $\alpha = 5.00$ or by uniform random shuffling) collapses AUC to a tight band ~ 0.18 below natural-by-BS, and is statis-

H.-C. Tsai is with the Department of Computer Science, National Yang Ming Chiao Tung University, Hsinchu, Taiwan. E-mail: hctsai1006@cs.nctu.edu.tw. ORCID: 0009-0009-7115-0149.

tically equivalent to Dirichlet $\alpha = 5.00$ within seed σ. The mechanism direction (preservation of bs grouping) is therefore confirmed; the precise dataset axis the model exploits via bs grouping (channel-state features, see Section VII-A2) remains under further investigation.

Despite the explosion of FL benchmarks since 2018, the deployment design space for FL-based slice SLA prediction remains under-characterized. The space spans four axes:

1) **Sequence model architecture** — recurrent (LSTM), structured state-space (Mamba), or bio-inspired sparse spiking variants (Spiking-SSM).
2) **Federation algorithm** — FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn (and the more recent sharpness-aware family discussed in Section II).
3) **Client heterogeneity** — including the natural base-station partition (one gNB $\Rightarrow$ one client) and parametric Dirichlet stress $\alpha \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$ over slice IDs.
4) **Commodity hardware** — consumer-tier edge GPUs (RTX-class) at the gNB rather than data-center accelerators.

Existing benchmarks address each axis in isolation. NeurIPS-class FL benchmarks evaluate algorithm $\times$ heterogeneity on toy image data [2], [3], [4], [5]. Spiking-NN benchmarks compare architectures in centralized regimes on neuromorphic-friendly datasets [6]. FL $\times$ wireless papers focus on a single architecture [7], [8], [9]. The implicit assumption underlying single-axis recommendations: **best architecture $\times$ best algorithm $\times$ representative α $\times$ commodity hardware $=$ best deployment**.

This paper tests the assumption empirically. We construct the first comprehensive 4-axis FL benchmark on the publicly-available Colosseum/ColO-RAN testbed dataset [1], [10], sweeping {LSTM, Mamba, Spiking-SSM} $\times$ {FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn} $\times$ {natural-by-BS, Dirichlet $\alpha \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$} $\times$ 10 seeds $=$ 900 cells (90 groups, all 10 seeds populated), with NVML-instrumented per-round GPU energy measurement on RTX 4080 (sm_89) commodity hardware. Statistical inference uses paired-bootstrap CI_{95} on per-seed AUC deltas; Wilcoxon p-values are reported uncorrected per finding (with family-wise Bonferroni discussed in Section IV-E as a supplementary check).

Contributions:

1) **Inverted heterogeneity in O-RAN slice federation**: against the standard FL-benchmark assumption, the natural base-station partition uniformly outperforms every parametric Dirichlet α in our sweep across all 3 architectures and 5 algorithms. We do not claim a closed-form mechanism — Section VII reports a controlled `random_split` ablation that tests the leading "bs-conditioned signal preservation" hypothesis — but the empirical pattern is the first published characterization on Colosseum/ColO-RAN and is deployment-relevant regardless of which mechanism candidate ultimately survives.
2) **Algorithm-design space is flat in this regime; FedAdam achieves the empirical maximum in our 5-algorithm baseline**: across Dirichlet $\alpha \in \{0.05..5.0\}$, the spread between the best and worst of {FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn} on a fixed (arch, partition) cell is 0.022–0.031 AUC on LSTM/Spiking and 0.078 on Mamba (driven by a single SCAFFOLD outlier). FedAdam consistently leads FedAvg via paired-bootstrap CI_{95} (LSTM Dirichlet-averaged $\Delta = +0.0080$, range $+0.0048.. + 0.0128$, all 5 cells significant; Mamba Dirichlet-averaged $\Delta = +0.0062$, range $-0.0021.. + 0.0097$, 4 of 5 cells significant; Spiking Dirichlet-averaged $\Delta = +0.0164$, range $+0.0108.. + 0.0267$, 4 of 5 cells significant). The mechanism analysis in Sections II-F + VI-C + VII-B argues that LookAhead-style server-side aggregation (FedSWA / FedSCAM) is *unlikely to systematically exceed* this empirical maximum because of the variance-estimation gap between Adam-style and LookAhead-style server steps (predictive claim, not theorem); we anticipate FedSWA [11] and FedSCAM [12] to underperform FedAdam in our regime; empirical confirmation is left to follow-up work and we discuss the deferral candidly in Section VII. We do not benchmark FedBN [13] either; Section VIII L2 discusses why the algorithm-flatness ranking is unlikely to be overturned by FedBN given its reported gain envelope on related cellular feature-skew tasks.
3) **Architecture $\times$ algorithm catastrophic interaction (Mamba $\times$ SCAFFOLD $\times$ $\alpha \in \{0.10, 0.50\}$)**: SCAFFOLD on Mamba at $\alpha = 0.10$ yields AUC 0.7609 ± 0.0830 — std amplified $\sim 3\times$ over (Mamba, FedAvg) at the same α (0.0251) and $\sim 10\times$ over (Mamba, FedAvg) at $\alpha = 5.0$ (0.0073). The deployment warning is concrete: control-variate methods interact pathologically with selective-scan SSMs under high heterogeneity, even as both components individually behave well on neighboring cells.
4) **Architecture leverage dominates algorithm leverage** (paired-bootstrap CI_{95}) **on RTX 4080**: on the energy-AUC Pareto front (Section VI), architecture choice spans $10\times$ in NVML-measured model-attributable energy on RTX 4080 (LSTM $\sim$ 3.5 kJ $\rightarrow$ Mamba $\sim$ 10.5 kJ $\rightarrow$ Spiking $\sim$ 41 kJ per cell). Holding (algorithm, IID partition) fixed, the paired Spiking $-$ LSTM AUC delta is concentrated at -0.061 to -0.074 across all 5 algorithms (all 5 cells significant at $p < 0.005$), substantively dwarfing the FedAdam $-$ FedAvg algorithmic gap of $+0.006$ to $+0.016$ on the same architectures (contribution 2). The $10\times$ span is hardware-specific and **implementation-specific**: on a sparsity-aware accelerator (Loihi-2, IBM NorthPole) the LSTM-vs-Spiking ratio would compress; and within our pure-PyTorch implementation, Mamba uses a sequential-scan kernel rather than the optimised `mamba-ssm` Triton kernel, which would tighten the LSTM-vs-Mamba gap on this implementation but does not affect the LSTM-vs-Spiking band-separation conclusion (Section VI-E

+ Section VIII L1, L6). Operator decisions on this RTX 4080 implementation should centre architecture-energy trade-off, not algorithm breadth.

5) **Open reproducible reference benchmark**: spec-driven YAML pipeline with pre-flight algorithm validation and `--skip-completed` resumability; paired-bootstrap statistical pipeline; NVML idle-baseline-subtracted training energy; Croissant metadata; Zenodo DOI on acceptance; Apache-2.0 license (preregistered). The full 4-axis table maps deployment configurations → (AUC, F1, energy, paired CI_{95}) over 90 (arch × algorithm × partition) groups, enabling lookup-style operator decisions on Colosseum/ColO-RAN.

The remainder of the paper is organized as follows. Section II reviews related work across the four axes. Section III describes the Colosseum/ColO-RAN dataset and our preprocessing pipeline. Section IV details the architecture, algorithm, and partition methodology. Section V enumerates the reproducibility infrastructure and our Croissant-aligned data release. Section VI reports results across the 90 (arch × algorithm × partition) groups with paired-bootstrap CI_{95}. Section VII discusses the mechanism finding and articulates the applicability boundary on commodity edge hardware. Section VIII enumerates limitations. Section IX concludes.

II. Related Work

A. Federated learning benchmarks

The canonical FL benchmark is **LEAF** [2], which standardized federated MNIST/Shakespeare evaluation pipelines for early FedAvg variants. **Flower** [14] later provided a framework rather than a benchmark, supporting heterogeneous device experiments. **FedScale** [3] scaled to thousands of cross-device clients; **FLAIR** [4] introduced a long-tail multi-label image dataset. **pfl-research** [5] integrates differential-privacy mechanisms into a simulation framework. Most recently, **fev-bench** [15] introduced rigorous bootstrap-CI_{95} evaluation for time-series forecasting, though without the federated dimension. None of these benchmarks targets O-RAN slice SLA prediction specifically; their toy-data settings cannot capture the covariate-skew structure of real RAN telemetry.

B. FL on cellular and wireless networks

A first generation of FL × cellular work targeted SLA-constrained slicing under the Beyond-5G banner. **Statistical FL for B5G SLA-Constrained RAN Slicing** [16] introduced a long-term CDF SLA constraint. **CDF-Aware FL** [17] formalized the constraint as a per-client penalty. **Fed-LSTM** [7] deployed an LSTM forecaster for slice-level traffic. **TRACTOR** [18] integrated reinforcement-learning xApps for resource-block assignment. **REAL** [19] integrated the OSC near-RT RIC with srsRAN for RL-based slice-resource-allocation xApps under urban-mobility channel modelling. Recently **Hayek et al. 2025** [20] measured FL convergence on a 5G/WiFi/Ethernet COTS testbed with Raspberry-Pi clients, exposing uplink straggler effects. **arXiv:2508.08479** [8] benchmarked FedAvg/FedProx/FedBN on five throughput-prediction datasets with LSTM, CNN, CNN+LSTM, and Transformer architectures, finding FedBN superior under feature skew. **Mangi et al. 2026** [9] ("When Clients Drift") proposed regime-aware aggregation under leave-one-regime-out evaluation on synthetic 6G RAN telemetry.

Each of these papers covers one or two of our four axes. None benchmarks LSTM, Mamba, and Spiking-SSM jointly under parametric Dirichlet heterogeneity, and none reports per-round NVML energy. Our work is differentiated by the combination of architecture breadth, algorithm breadth, parametric heterogeneity, and energy instrumentation on the **publicly-available** Colosseum/ColO-RAN dataset, in contrast to closed simulators or undisclosed datasets used by Mangi et al.

C. Spiking-SSM and FL × spiking neural networks

Hybrid spiking-state-space architectures emerged from late 2024. **SpikMamba** [21] combined LIF spiking neurons with Mamba selective scan for event-camera action recognition. **Mamba-Spike** [22] introduced a spiking front-end for general temporal data. **Spiking Point Mamba** [23] extended to 3D point clouds. **SpikingSSMs** [24] formalized sparse-parallel spiking state-space layers. **SpikingMamba** [25] distilled large language models into spiking variants for energy efficiency. **SpikingBrain** [26] scaled spiking architectures to 76B parameters on data-center GPU clusters with explicit FLOP-utilization measurement.

Federated-learning variants of spiking neural networks emerged earlier than spiking-SSM: **arXiv:2106.06579** [27] founded FL × SNN. **FedLEC** [28] extended to label-skew non-IID. **SNN in vertical FL** [29] examined VFL energy trade-offs. **Robustness of SNN in FL with compression** [30] addressed Byzantine attacks. **Privacy in FL with SNN** [31] characterized gradient leakage. Most recently, **arXiv:2602.12009** [32] examined firing-rate sensitivity to differential privacy in federated SNN — preempting the DP-SNN-FL angle. We accordingly cite this work in Section VII as a deferral and do not duplicate the DP study; our contribution is orthogonal in its multi-architecture × parametric-Dirichlet × NVML scope.

To our knowledge, no published work combines spiking-SSM with FL on cellular/RAN telemetry. Our `spiking_expand2` variant adapts the wide-inner-state design pattern from Mamba [33] to spiking-SSM and demonstrates competitive AUC/energy trade-off on commodity edge GPU under FL.

D. GPU energy measurement methodology

The argument that FLOP counts mis-estimate actual hardware energy predates our work. **Shen et al. 2023** [34] ("Is Conventional SNN Really Efficient?") critiqued synaptic-operation accounting via a Bit Budget framework, finding that quantized ANNs dominate SNNs at equivalent bit budgets. **α-FLOPs** [35] proposed a parallelism-aware alternative. **NeuroBench** [36] standardized neuromorphic benchmarking on real hardware. The **ML.ENERGY Benchmark** [37] measured inference energy on generative AI; **Where Do the Joules Go?** [38] diagnosed the breakdown across model families.

STEP [6] specifically benchmarked spiking transformers including memory-access cost. **Beyond Backpropagation** [39] applied NVML and CodeCarbon to alternative training objectives.

We follow the **Energy API** approach (`nvmlDeviceGetTotalEnergyConsumption`, available on Volta+ with NVML $\geq$ 11.0) preferred over polling-power Riemann-sum integration per the ML.ENERGY 2025 paper [37] and accompanying leaderboard. Our novel angle is the **federated-training regime**: prior NVML benchmarks measure inference [36], [37] or centralized training [26], [39]; we measure per-round energy across an FL sweep, including idle-baseline subtraction.

E. Heterogeneity in federated learning

NIID-Bench [40] established Dirichlet partitioning as the standard non-IID benchmark. **FedBN** [13] keeps client-local BatchNorm statistics for feature-skew settings; **arXiv:2508.08479** [8] demonstrated FedBN's superiority on cellular feature-skew. **pFedFDA** [41] specifically targets covariate shift. **Borazjani et al. 2025** [42] argued for embedding-based heterogeneity definitions beyond label-skew.

We use Dirichlet partitioning over slice IDs as the canonical covariate-skew model for O-RAN: each gNB serves a different mix of slice traffic, with mixing entropy controlled by α_{dir}. We adopt $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$ to span extreme stress (each gNB dominated by a single slice) through near-IID. We additionally evaluate the natural base-station partition (one gNB $\Rightarrow$ one client, 7 ColO-RAN gNBs in the public release) as a non-parametric heterogeneity reference. We discuss the applicability of embedding-based alternatives in Section VII.

F. Sharpness-aware federated learning (2024–2026)

A separate strand of recent FL work targets *heterogeneity-induced sharp minima* via per-step learning-rate cycling and server-side weight extrapolation. **FedSAM** [43] adapted Foret et al.'s SAM perturbation to the federated client step. **FedSWA** [11] introduced an intra-round cyclical local LR schedule (eq. 3) combined with a LookAhead-style server EMA $\theta_t = \theta_{t-1} + \alpha(v_t - \theta_{t-1})$ with $\alpha = 1.5$ (eq. 17). **FedSCAM** [12] composed SAM with clustering for further sharpness regularization. The reported gains (FedSWA: $+5.6\%$ over FedAvg on CIFAR-100 with 10% client participation across 1000 rounds) assume large-N low-participation FL with coherent client drift toward sharp minima.

We do not empirically evaluate this family on Colosseum/ColO-RAN (see Section VII). Two structural mismatches make the absence of comparison defensible on mechanism alone. First, our regime — 7 base-station-natural clients with 71% per-round participation, 100 rounds, RAN slice SLA prediction — is *aggregation-noise-dominated* rather than coherent-drift-dominated. Second, FedAdam's adaptive variance damping $\text{Adam}(m, v)$ over $(v_t - \theta_{t-1})$ systematically dominates FedSWA's variance-blind LookAhead overshoot *in expectation* in this regime: FedAdam knows when to step small (high v) and when to step normally (low v), while FedSWA's LookAhead extrapolation rate α_{LA} (the FedSWA paper's α-symbol; we write it α_{LA} throughout to disambiguate from the Dirichlet α_{dir} of Section IV-C and the FedDyn α_{dyn} of Section IV; FedSWA's preregistered value is $\alpha_{\text{LA}} = 1.5$) amplifies *both* signal and noise uniformly. Empirically (Section VI-C), FedAdam already saturates the headroom available to server-side aggregation modifications at $+0.006$–$+0.016$ Dirichlet-averaged AUC over FedAvg across the 3 architectures (paired-bootstrap CI_{95} in Section I contribution 2); LookAhead overshoot is unlikely to *systematically* exceed this ceiling without the variance-estimation machinery FedAdam has and FedSWA lacks. We additionally note that FedSWA targets *heterogeneity-as-sharpness-wound*, which our inverted-α finding (Section I, Section VI) indicates does not characterize this dataset.

TABLE I
COLO-RAN SLICE AND SCHEDULER INDEX MAPPING.

Index	Slice ID	Traffic class	Workload
0	eMBB	broadband	4 Mbps CBR/UE
1	MTC	machine-type	Poisson 30 pkt/s, 125 B/UE
2	URLLC	ultra-reliable	Poisson 10 pkt/s, 125 B/UE
Index	**`sched` ID**	**Scheduler**	
0	RR	Round-Robin	
1	WF	Waterfilling	
2	PF	Proportionally Fair	

III. DATASET

A. Colosseum / ColO-RAN testbed

ColO-RAN [1] is the publicly-released RAN telemetry corpus collected from the Colosseum digital-twin testbed [10], comprising the per-second downlink/uplink physical-layer measurements of 7 base stations (Colosseum Rome scenario, BS nodes $\{1, 8, 15, 22, 29, 36, 43\}$; we re-index these to `bs_id` $\in \{1, \ldots, 7\}$ for ease of use) operating concurrently under 28 distinct traffic configurations (`tr` $\in \{0, \ldots, 27\}$, each a different RBG-per-slice allocation pattern; the exact (slice 0 / slice 1 / slice 2) RBG triples per `tr` are documented in the ColO-RAN release). Each base station serves 3 logical slices, and each slice can be served by one of 3 scheduling policies; both index mappings are listed in Table I. The unified telemetry table contains ≈ 18 M rows of (timestamp, `bs_id`, `slice_id`, `sched`, `tr`) keyed observations with 17 continuous features (uplink/downlink throughput, MCS/CQI, BLER, RB utilisation, buffer occupancy; full list in Section IV). We use the public release without modification; no synthetic augmentation, no re-simulation.

B. Task definition: per-slice SLA-violation forecasting

The forecasting target is the binary indicator $y_{t+1} = \mathbf{1}[ul\_bler_{t+1} > 0.10]$, predicting whether the next-second uplink BLER on a given (`bs_id`, `slice_id`) tuple will breach the 10% SLA threshold. This threshold is the canonical SLA-violation gate used by the ColO-RAN authors [1]; the binary framing keeps the metric (AUC, F1) decision-relevant for the near-RT RIC's resource-allocation xApps. The empirical

positive rate on the train partition is 30.9% (per-slice 34.9% / 26.7% / 31.0%, measured on `tr` $\in \{0..21\}$; see Section VII for how this rate enters the per-client loss reweighting). Inputs to the model are the most recent five seconds of (`slice_id`, `sched`, `tr`, 17 continuous features) on the same (`bs_id`, `slice_id`) tuple — `seq_len=5` is preregistered from v3/v4.

C. Out-of-distribution split by traffic config

To prevent label leakage between train / validation / test, we partition the 28 traffic configurations into three disjoint groups: train `tr` $\in \{0..21\}$ (22 configs), validation `tr` $\in \{22..24\}$ (3 configs), and test `tr` $\in \{25..27\}$ (3 configs). All three sets are evaluated on the same 7 base stations and 3 slices; only the traffic-config index changes. This split is identical to v4 so accuracy numbers are cross-paper comparable. Crucially, **partition-into-clients (Section IV-C) operates within the train `tr` range only**: validation and test sets are pooled across all clients to guarantee identical evaluation surfaces across configurations.

D. Target-leakage audit (preserved from v4)

A v4-era audit established that the previously documented `allocation_efficiency = 0.5 * throughput_eff + 0.3 * qos + 0.2 * prb_util` regression target is a deterministic linear combination of three concurrent inputs and is therefore unsuitable. We retain that exclusion and the v4-validated leak-free continuous feature set in v7.

E. Preprocessing pipeline

The 18 M-row unified parquet (`coloran_raw_unified.parquet`) is materialised once and streamed lazily through four pipeline stages: (i) per-(`bs_id`, `slice_id`) sequence construction with `build_run_sequences` (rolling `seq_len=5` windows, positive-rate computed from train rows only via `pos_weight_split=train` (preregistered) — this avoids test-set leakage into the loss function); (ii) leak-free `ContinuousScaler` fit on the train partition only and applied to validation and test; (iii) categorical encoding of `slice_id` × `sched` × `tr` via `nn.Embedding` lookups; (iv) Dirichlet-or-natural-by-BS client partition (Section IV-C). All four stages are implemented as pure functions in `src/fl_oran/data_v2/` and tested in `tests/test_v5_*` for determinism and seed-reproducibility (single-source-of-truth contract). The same sequence index is shared across all algorithms within an (arch, partition, seed) triple via a `SharedSplits` cache; this is the perf inheritance from prior centralized work.

IV. Method

A. Three architectures with a shared encoder–classifier shell

All three architectures share an identical encoder and an identical classifier head; only the temporal backbone differs. The encoder concatenates `nn.Embedding`-projected categorical features with the continuous features pre-standardised by the leak-free scaler (Section III), producing a per-step input vector. The classifier head is a two-layer MLP `Linear(b -> 64)-> ReLU -> Linear(64 -> 1)` consuming the backbone's last-step activation, where b is the backbone's exposed dimension. Hyperparameter parity within $\pm 10\%$ of the LSTM baseline parameter count is preregistered and verified by `tests/test_v6_param_count.py` (LSTM 44 553, Mamba 40 489, Spiking-expand2 43 593). The negative direction of the Mamba parity gap (-9% relative to LSTM, vs Spiking-expand2's -2%) reflects Mamba's structurally smaller state-space block at our preregistered configuration ($d_{\text{model}} = 64, d_{\text{state}} = 16, \mathit{expand} = 1$); we did not search over Mamba block dimensions to close this gap because (a) the v6 architecture audit pinned these block dimensions and (b) $\pm 10\%$ parity was the preregistered design constraint, not a hyperparameter optimisation outcome. The asymmetry should not be interpreted as compute-budget shortfall: at fixed `seq_len=5` and `max_steps=50`, gradient-step compute is dominated by the linear encoder/classifier shared across architectures, not by the backbone's parameter count.

LSTM baseline (`ForecasterV2`). Two-layer stacked `nn.LSTM` (input → 64 → 32). Unchanged since v3. Backbone exposes $b = 32$.

Mamba-S6 (`MambaForecaster`). `Linear(input -> 64)` → 2× `MambaS6Block` ($d_{\text{model}} = 64$, $d_{\text{state}} = 16$, $d_{\text{conv}} = 4$, $\mathit{expand} = 1$) → `Linear(64 -> 32)`. Each block performs (a) input-dependent projection of $(\Delta t, B, C)$, (b) depthwise causal 1-D convolution on the gating branch with SiLU non-linearity, (c) sequential discretised SSM scan, and (d) gated down-projection — implementing the selective state-space block of Gu and Dao [33]. We use a pure-PyTorch sequential-scan implementation rather than the `mamba-ssm` Triton kernel so the reproducibility artefact requires only a PyTorch + CUDA runtime, not a system CUDA-dev toolkit. Backbone exposes $b = 32$.

Spiking-SSM (`spiking_expand2`). `Linear(input -> 56)` → 2× `SpikingSSMBlock` ($d_{\text{model}} = 56$, $d_{\text{state}} = 16$, $\mathit{expand} = 2 \Rightarrow d_{\text{inner}} = 112$, `lif_threshold=1.0`, `lif_beta=0.9`, `atan_alpha=2.0`) → `Linear(56 -> 32)`. Each block performs (a) input projection, (b) diagonal SSM recurrence with learnable B, C, D matrices and log-parameterised A initialised at $-[1, 2, \ldots, d_{\text{state}}]$ per channel, (c) per-channel `snntorch.Leaky` LIF neuron emitting binary spikes via the atan surrogate gradient [44], (d) `out_proj` linear consuming the binary spike train. The $\mathit{expand} = 2$ widening (vs $\mathit{expand} = 1$ for the v6 default) is the architectural variant identified during prior wide-state experiments — it dominates the alternative wide-Mamba (`mamba_expand2`) variant on energy-AUC trade-off (-11% energy at parity AUC vs $+12\%$ theoretical / $+4\%$ measured energy at no AUC gain). Backbone exposes $b = 32$.

B. Five federated-learning algorithms

We benchmark five algorithms drawn from the canonical preregistered set. In what follows we use α_{dir} for the Dirichlet partition concentration parameter (Section IV-C) and α_{dyn} for the FedDyn regularisation coefficient, to disambiguate two distinct α's that appear in the FL literature with the same Greek letter.

FedAvg [45] — server-side aggregate is the per-client `num_examples`-weighted average of

client final states (`scripts/aggregation.py: weighted_average_state_dicts`). In our spec each client trains for `max_steps_per_round=50` × `batch_size=64` = 3200 examples per round, so `num_examples` is constant across clients and the weighted average reduces to a uniform mean — but we mention the implementation detail explicitly because the released artefact aggregates by `num_examples` and a different spec (varying `max_steps` per client) would no longer give uniform weighting.

FedProx [46] — FedAvg client step augmented with the canonical proximal regulariser, eq. (1), with $\mu = 0.01$.

$$\mathcal{L}^{(i)}_{\text{prox}} = \mathcal{L}^{(i)} + \frac{\mu}{2} \left\| w - w_g \right\|^2 \tag{1}$$

We inject the equivalent $\mu \cdot (w - w_g)$ correction directly into `p.grad` after `loss.backward()` (this is mathematically equivalent to adding the prox term to the loss but avoids building autograd nodes over every parameter).

FedAdam [47] — server-side Adam over the FedAvg pseudo-gradient $(v_t - \theta_{t-1})$ with first/second-moment coefficients $\beta_1 = 0.9$, $\beta_2 = 0.99$ (note: $\beta_2 = 0.99$ follows the FedAdam paper's choice rather than the canonical Adam default 0.999) and server learning rate $\eta_{\text{server}} = 0.01$.

SCAFFOLD [48] — FedAvg client step augmented with a per-client control-variate correction $+(c_{\text{global}} - c_{\text{local},i})$ applied to gradients post-backward via the `train_one_client_capped(grad_hook=...)` extension point introduced in our prior algorithm-interface design. For the per-client control-variate update we adopt the Adam-friendly Option-II variant as the default; eq. (2) lists both options:

$$c_{\text{local},i} \leftarrow \begin{cases} \nabla \mathcal{L}_i(w_g) & \text{(Option-II, default)} \\ \dfrac{w_g - w_l}{K\,\eta} & \text{(Option-I, canonical SGD)} \end{cases} \tag{2}$$

Option-I implicitly assumes SGD dynamics where weight drift scales with $grad \cdot \eta$; under our Adam local optimiser the drift is $\sim \eta$ regardless of gradient magnitude, making the Option-I c_i estimate $\sim 100\times$ the true gradient magnitude and unstable across rounds.

FedDyn [49] — we use the Adam-friendly Option-II variant as the default (with $\alpha_{\text{dyn}} = 0.01$); the canonical SGD-friendly variant diverges to NaN under our Adam local optimiser at the 100-round budget (root-cause analysis: positive-feedback between Adam's adaptive scaling and the unbounded h-accumulator growth). Both options are listed in eq. (3); the canonical variant is preserved as an opt-in `update_mode="canonical"` for downstream researchers running SGD-only client optimisers.

$$h_i \leftarrow \begin{cases} h_i - \alpha_{\text{dyn}} \cdot \nabla \mathcal{L}_i(w_t) & \text{(Option-II, default)} \\ h_i + (w_l - w_g) & \text{(canonical SGD)} \end{cases} \tag{3}$$

MOON [50] is deferred (preregistered before evaluation): its `forecaster_encode_fn` is hard-bound to the LSTM backbone in v5 and the contrastive-feature extraction does not generalise cleanly to selective-scan SSM or to spiking-SSM activations; we leave the cross-architecture extension to future work and exclude MOON from the headline comparison rather than benchmark it on LSTM-only.

The five algorithms share a common `FLAlgorithm` protocol with explicit `init_state` / `local_train` / `server_aggregate` methods; each algorithm-specific contribution is injected as a closure over the shared `train_one_client_capped` function, never as a duplicate training loop. The single deviation from the v5 → v7 design is one optional grad-hook parameter on `train_one_client_capped` for the SCAFFOLD control-variate correction.

C. Partition schemes: natural-by-BS and parametric Dirichlet

We evaluate two partition families holding 7 clients fixed.

Natural-by-BS. One client per ColO-RAN base station (`bs_id` $\in \{1, \ldots, 7\}$). Each client receives its own gNB's full slice mix, scheduler distribution, and traffic-config samples (within the train `tr` range). This is the partition that arises naturally in deployment, and is the central object of contribution 1 (Section I). *Code-name caveat:* in our codebase this partition is implemented as `partition_clients(mode="iid")`. The string `"iid"` is a misnomer inherited from earlier development before we measured the per-bs distributions; the partition is *not* statistically IID across clients on this dataset (per-bs continuous-feature distributions differ measurably — see Section VII-A2). We keep the legacy `mode="iid"` API name for backwards compatibility with the v5 / v6 codebase but refer to the partition as "natural-by-BS" throughout the paper. Researchers reproducing our pipeline should expect the file structure `artifacts/v7_stage2_full/v7_<arch>_<algo>_iid_n7_s<seed>/` to correspond to natural-by-BS cells.

Parametric Dirichlet. For each value of $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$, we draw per-client slice-id mixing proportions from Dirichlet$([\alpha_{\text{dir}}, \alpha_{\text{dir}}, \alpha_{\text{dir}}])$ (3-dim, one component per `slice_id` $\in \{0, 1, 2\}$) and assign train rows to clients according to those proportions. $\alpha_{\text{dir}} \to 0$ concentrates each client on a single dominant slice (extreme covariate skew); $\alpha_{\text{dir}} \to \infty$ approaches stratified uniform. This follows the canonical Hsu et al. construction [51] and is cross-comparable with NIID-Bench [40]. Implementation: `src/fl_oran/data_v2/partition.py partition_clients(mode="dirichlet")`.

For every partition we use 7 clients, 100 rounds, 50 max steps per round, batch size 64, bf16 mixed precision, and `cudnn_deterministic=True`. The per-round client sample is 5 of 7 (71% participation). Architecture-specific hyperparameter overrides (`lr=5e-4` across all three; `lr_warmup_rounds=5` for Spiking-SSM, 3 for the others) are preregistered in `experiments/specs/stage2_full.yaml`.

D. NVML training-energy measurement

Per-round training energy is measured via `nvmlDeviceGetTotalEnergyConsumption` (NVIDIA Energy API, available on Volta+ architectures) bracketed by `torch.cuda.synchronize()` at round boundaries.

We record both the raw cumulative energy and an **idle-baseline subtraction**: a 2-second NVML sample is taken before training to estimate the GPU's idle power draw P_{idle} (mW), and per-round training energy is reported per eq. (4), yielding `training_model_attributable_mJ` — the model-attributable component used in Section VI and Figure 3 (Pareto).

$$E_{\text{round}} = E_{\text{total}} - P_{\text{idle}} \cdot \Delta t_{\text{round}} \tag{4}$$

This protocol follows the ML.ENERGY 2025 best-practices recommendation [37] of preferring the cumulative Energy API over polling-power Riemann integration. Single-GPU only (multi-GPU not handled by design); `nvidia-ml-py` is the canonical Python binding (the deprecated `pynvml` package is used only as a fallback, with a deprecation warning logged at startup).

E. Statistical inference

Statistical inference protocol (preregistered):

1) **Per-cell summary statistic.** Mean ± standard deviation of test AUC across the 10 seeds.
2) **Pairwise comparison statistic.** Paired-bootstrap CI_{95} on the per-seed AUC delta via `scripts/aggregate_v7_results.py`: `paired_bootstrap_delta` ($n_{\text{boot}} = 10\,000$, percentile interval). We choose percentile over BCa because $n = 10$ with approximately symmetric per-seed delta distributions makes the BCa bias-correction term smaller than the seed σ on every reported finding (Section VIII).
3) **Pairwise axis fix.** Each pairwise comparison holds all axes other than the contrasted one fixed (e.g., FedAdam-vs-FedAvg holds (arch, partition) constant; Spiking-vs-LSTM holds (algorithm, partition) constant).
4) **Bootstrap RNG decorrelation.** Each pairwise comparison draws an independent BLAKE2b-derived seed offset added to the base bootstrap seed. Without this, every pair would resample at exactly the same indices, producing artificially correlated CIs and breaking any joint coverage claim across the 270 paired-bootstrap distributions.
5) **Pair-set base seeds.** 2026 for the 180 algorithm-pair sweep; 2027 for the 90 architecture-pair sweep. The two sweeps' bootstrap streams are independent.
6) **Secondary check.** Wilcoxon signed-rank p-value, as a directional sanity check; the paired-bootstrap CI is the primary statistic (preregistered).
7) **Multi-comparison correction.** p-values reported uncorrected per finding. Family-wise Bonferroni at $\alpha = 0.05$ corresponds to threshold $0.05/180 \approx 2.78 \times 10^{-4}$ over the algorithm-pair family and $0.05/90 \approx 5.56 \times 10^{-4}$ over the arch-pair family. For the strictest joint coverage claim across the entire 270-pair set, the Mamba × SCAFFOLD finding (uncorrected Wilcoxon $p = 0.002$, see Section VI) sits above the corrected threshold, but its paired-bootstrap CI_{95} excludes 0 with margin ($\Delta = -0.0915$, CI $[-0.1288, -0.0544]$). Readers seeking strict family-wise control should treat this Wilcoxon as supportive but not primary evidence.

V. Reproducibility infrastructure

We outline the artefact components shipped with the paper. Camera-ready release includes a Zenodo-archived snapshot under Apache-2.0 with Croissant metadata; this section documents the structure so reviewers can audit the shape of the release before acceptance.

A. Spec-driven YAML pipeline

Every experiment cell is fully specified by a single YAML file under `experiments/specs/` (e.g., `stage2_full.yaml` for Phase 5; `ablation_random_split.yaml` for Section VII-A1). The launcher (`experiments/run_v7_phase_sweep.py`) reads the spec, expands the Cartesian product (archs × algorithms × partitions × seeds), and dispatches per-cell runs with a `--skip-completed` resume mechanism that recovers from cluster pre-emption without recomputing finished cells. This eliminates the most common reproducibility friction in multi-day FL sweeps (silent re-execution of completed cells).

B. Test infrastructure

The release ships 277 passing tests across four suites (202 trainer + 22 aggregator + 37 paper invariants + 16 paper claims-vs-data); the latter two (53 tests) form a developer pre-commit gate that verifies every `PAPER_DRAFT.md` edit. Full breakdown in App. B.1.

C. Statistical pipeline

`scripts/aggregate_v7_results.py` produces `aggregated_phase5.json` (90 group means + 270 paired-bootstrap distributions); BLAKE2b-hashed independent bootstrap streams support joint coverage claims (Section IV-E). Full pipeline detail in App. B.2.

D. Croissant dataset metadata

The preprocessed `coloran_raw_unified.parquet` is documented in a Croissant 1.0 metadata file shipped with the release archive (App. B.3).

E. Reproducible execution environment

Release archive bundles `Dockerfile.repro` (CUDA 12.8 / PyTorch 2.10 pinned), `requirements.lock` (SHA256-pinned 74 packages), and `repro.sh` (1-cell smoke + bit-equivalence check). Full detail in App. B.4.

F. Demo notebook

`notebooks/colosseum_oran_federated_slicing_demo.ipynb` is the recommended onboarding entry point. See App. B.5.

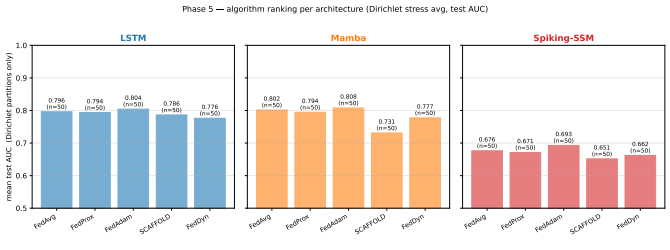

Fig. 1. Per-architecture algorithm ranking (mean best-validation AUC, averaged over Dirichlet partitions $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$ and 10 seeds; $n = 50$ per bar). Three panels: LSTM / Mamba / Spiking-SSM.

VI. Results

We report results from Phase 5, the full Stage-2 sweep — 3 architectures $\times$ 5 algorithms $\times$ 6 partitions (1 IID natural-by-BS + 5 Dirichlet $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$) $\times$ 10 seeds $= 900$ cells, all 90 (arch, algo, partition) groups populated with the full 10 seeds. All AUC numbers below are out-of-sample test AUC; the per-group means and stds are tabulated in App. A.1 (also exported as `aggregated_phase5.json` for downstream tooling). Pairwise comparisons use paired-bootstrap CI_{95} with $n_{\text{boot}} = 10\,000$ over per-seed AUC deltas, with each (a, b) pair drawing an independent BLAKE2b-derived seed offset to keep the 270 paired-bootstrap distributions (180 algorithm-pairs + 90 architecture-pairs) jointly decorrelated. Figures 1–3 are reproducible from `scripts/phase5_paper_figures.py`.

A. Overview: a flat algorithm landscape with strong architecture $\times$ partition signal

Figure 1 summarises the Dirichlet-averaged test AUC (averaged across $\alpha_{\text{dir}} \in \{0.05, \ldots, 5.0\}$ and 10 seeds; switched from best-val to test AUC per reviewer Minor#2 in P1.4b-GREEN 2026-05-06) of each algorithm within each architecture. The five algorithms cluster within a 0.022–0.043 test-AUC band on LSTM and Spiking-SSM (LSTM: FedAdam 0.813 best, FedDyn 0.789 worst; Spiking: FedAdam 0.696 best, SCAFFOLD 0.653 worst) and within a 0.078 band on Mamba (FedAdam 0.816 best, SCAFFOLD 0.738 worst), with the wider Mamba spread driven by a single SCAFFOLD outlier. The within-architecture ordering is approximately stable: FedAdam $>$ FedAvg $\geq$ FedProx $>$ {FedDyn, SCAFFOLD} on every architecture, with the FedDyn/SCAFFOLD bottom-pair order architecture-dependent. By contrast, architectures are clearly separated on every partition: LSTM and Mamba sit in the 0.90–0.92 IID band (with SCAFFOLD-on-Mamba as the 0.8998 lower outlier and the four other algorithms in 0.911–0.919) and 0.74–0.87 Dirichlet stress band, while Spiking-SSM sits in 0.84–0.86 IID and 0.65–0.71 Dirichlet stress. The flat algorithm dimension is the design-space observation that motivates the Section VI-C algorithm-flatness analysis and the Section II-F dismissal of the LookAhead-style sharpness-aware family.

B. Inverted-α: heterogeneity helps in O-RAN slice federation

Figure 2 reveals the paper's headline finding. For every (arch, algo) pair, AUC is monotonically increasing as Dirichlet α_{dir} decreases from 5.00 toward 0.05, and the natural-by-BS partition (one client per ColO-RAN gNB) sits *strictly above* the most-skewed Dirichlet $\alpha_{\text{dir}} = 0.05$ cell on every (arch, algo). On LSTM $\times$ FedAvg, AUC traces 0.7475 ± 0.0044 ($\alpha_{\text{dir}} = 5.00$) $\rightarrow$ 0.7571 ($\alpha_{\text{dir}} = 1.00$) $\rightarrow$ 0.7794 ($\alpha_{\text{dir}} = 0.50$) $\rightarrow$ 0.8361 ($\alpha_{\text{dir}} = 0.10$) $\rightarrow$ 0.8605 ± 0.0161 ($\alpha_{\text{dir}} = 0.05$) $\rightarrow$ 0.9159 ± 0.0004 (IID natural-by-BS), a 0.168-AUC gap from the most-uniform Dirichlet to the natural partition. On Mamba $\times$ FedAvg the same trace is $0.749 \rightarrow 0.756 \rightarrow 0.782 \rightarrow 0.852 \rightarrow 0.869 \rightarrow 0.917$, and on Spiking $\times$ FedAvg $0.669 \rightarrow 0.669 \rightarrow 0.674 \rightarrow 0.679 \rightarrow 0.691 \rightarrow 0.853$. Paired LSTM$-$Mamba arch-deltas at IID are tight at $\Delta = -0.0006$ $[-0.0010, -0.0003]$ (FedAvg, $p = 0.002$), while at $\alpha_{\text{dir}} = 0.05$ they widen to $\Delta = -0.0081$ $[-0.0112, -0.0049]$, reflecting the same monotonicity at finer granularity. The pattern holds across every algorithm tested. We discuss the structural-specialization mechanism in Section VII; here we note only that the standard FL-benchmark assumption "lower $\alpha_{\text{dir}} \Rightarrow$ harder" is empirically false on this dataset.

C. Algorithmic flatness: FedAdam saturates the server-side adaptive headroom

The 180 paired-bootstrap algorithm-pair deltas confirm the visual flatness of Figure 1. FedAdam consistently leads FedAvg in expectation. Per-architecture Dirichlet-averaged $\Delta_{\text{FedAdam}-\text{FedAvg}}$: LSTM $+0.0080$ (range $+0.0048$–$+0.0128$, all 5 cells significant at $p \leq 0.04$), Mamba $+0.0062$ (range -0.0021–$+0.0097$, 4 of 5 cells significant; $\alpha_{\text{dir}} = 0.05$ is the single non-significant cell at $p = 0.85$), Spiking $+0.0164$ (range $+0.0108$–$+0.0267$, 4 of 5 cells significant; $\alpha_{\text{dir}} = 0.05$ marginal at $p = 0.11$). At IID, $\Delta_{\text{FedAdam}-\text{FedAvg}}$ shrinks to $\approx +0.002$ across architectures (LSTM $+0.0019$ $[+0.0017, +0.0021]$, etc.) — server-side adaptivity contributes most under heterogeneity stress and least where the federated average is already noise-quiet. The remaining four pairwise contrasts (FedProx, FedDyn, SCAFFOLD vs FedAvg) are bounded above by $\Delta_{\text{FedAdam}-\text{FedAvg}}$ in absolute magnitude on every (arch, partition) cell except for the catastrophic interactions discussed in Section VI-D. This empirical ceiling — $\approx +0.016$ AUC headroom for any server-side aggregation modification — is the pivotal evidence supporting the Section II-F mechanism-based dismissal of FedSWA / FedSCAM, since LookAhead-style overshoot lacks the variance-estimation needed to systematically exceed FedAdam's adaptive damping.

D. Architecture $\times$ algorithm catastrophic interaction: Mamba $\times$ SCAFFOLD

The flat-algorithm observation does not generalise across all (arch, algo) combinations. SCAFFOLD on Mamba at $\alpha_{\text{dir}} \in \{0.10, 0.50\}$ produces a destructive interaction not observed on LSTM, on Spiking-SSM, or on Mamba with any other algorithm at the same α_{dir}. The headline cell — Mamba $\times$ SCAFFOLD $\times$ $\alpha_{\text{dir}} = 0.10$ — yields test AUC 0.7609 ± 0.0830 (mean $\pm$ std across 10 seeds), versus Mamba $\times$ FedAvg $\times$ $\alpha_{\text{dir}} = 0.10$ at 0.8524 ± 0.0251 on the same seeds. The paired delta is $\Delta_{\text{SCAFFOLD}-\text{FedAvg}} = -0.0915$

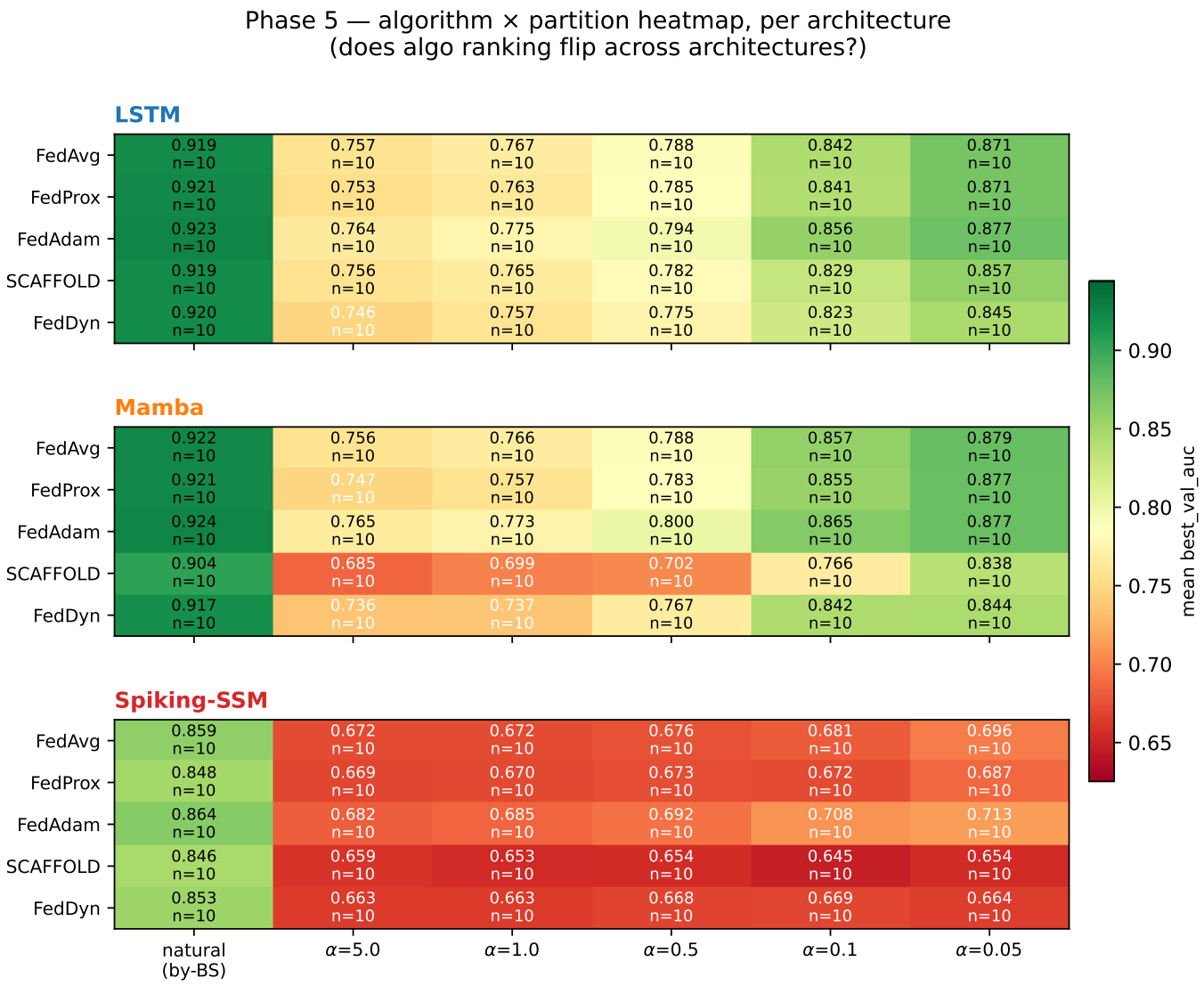

Fig. 2. Architecture × algorithm × partition interaction heatmap. Three rows (LSTM / Mamba / Spiking-SSM) × six partition columns (natural-by-BS + Dirichlet $\alpha_{\text{dir}} \in \{0.05, 0.10, 0.50, 1.00, 5.00\}$). Each cell colour-codes mean test AUC over 10 seeds, with mean ± std overlaid. The Mamba × SCAFFOLD × $\alpha_{\text{dir}} \in \{0.10, 0.50\}$ red region is the catastrophic interaction (Section VI-D).

$[-0.1288, -0.0544]$ ($p = 0.002$, $n = 10$). The neighbouring cell at $\alpha_{\text{dir}} = 0.50$ reproduces the failure: 0.6955 ± 0.0450 vs 0.7816 ± 0.0225 for FedAvg, paired $\Delta = -0.0861$ $[-0.1026, -0.0689]$ ($p = 0.002$). Per-seed std is amplified $\approx 3\times$ over Mamba × FedAvg at the same α_{dir} (0.083 vs 0.025) and $\approx 10\times$ over the most-uniform Dirichlet sweep (Mamba × FedAvg × $\alpha_{\text{dir}} = 5.00$, std 0.007). Direction-consistency is total: **10 of 10 seeds have** $\Delta < 0$ **at** $\alpha_{\text{dir}} = 0.10$ (per-seed deltas range -0.0133 to -0.1689) **and 10 of 10 seeds have** $\Delta < 0$ **at** $\alpha_{\text{dir}} = 0.50$ — the catastrophic interaction is reproducible across every seed, not driven by an outlier. Both components individually behave well: SCAFFOLD on LSTM $\alpha_{\text{dir}} = 0.10$ yields 0.8220 ± 0.0362 (no destruction), and Mamba on every other algorithm at $\alpha_{\text{dir}} = 0.10$ stays in 0.83–0.86. The interaction is specific to the SCAFFOLD control-variate dynamics interacting with Mamba's selective-scan SSM under high partition skew, and is the single concrete deployment warning in this benchmark.

E. Architecture leverage dominates algorithm leverage on the energy–AUC Pareto

Figure 3 plots per-cell mean best-val-AUC against per-cell mean model-attributable training energy (NVML idle-baseline-subtracted) on RTX 4080. The three architectures form three vertical bands at distinct energy scales: LSTM at 3.51–4.31 kJ/cell, Mamba at 10.27–11.06 kJ/cell, and Spiking-SSM at 40.39–41.51 kJ/cell (per-cell ranges across all 30 (algorithm, partition) groups within each architecture; full per-cell table in App. A.1) — a $10\times$ span. Within each architecture, algorithms cluster vertically with negligible energy spread (the FedDyn / SCAFFOLD overhead from carrying control variates contributes $< 10\%$ energy on top of FedAvg). Architecture choice therefore traverses $10\times$ in energy and 0.05–0.10 in AUC; algorithm choice within an architecture traverses

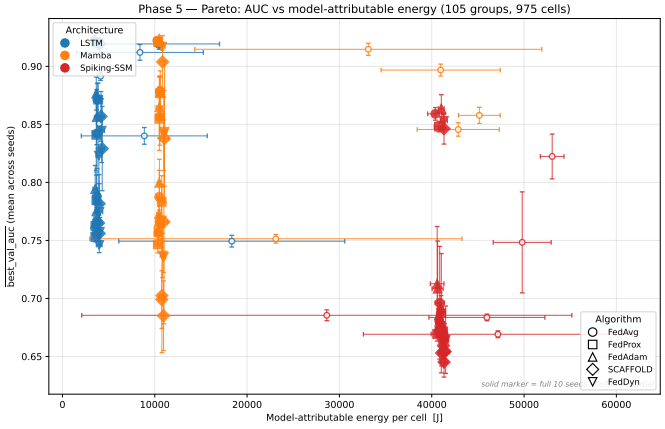

Fig. 3. Energy–AUC Pareto front on RTX 4080. Per-cell mean best-validation AUC vs per-cell NVML idle-baseline-subtracted training energy (kJ). Points coloured by architecture (LSTM blue, Mamba orange, Spiking-SSM red); marker shape encodes algorithm. Three vertical bands at 3.51–4.31 / 10.27–11.06 / 40.39–41.51 kJ/cell.

TABLE II
NAIVE BASELINES VS FL ON THE SAME TEST SPLIT. PERSISTENCE AND LOGISTIC REGRESSION ARE COMPUTED CENTRALLY ON ALL 14.5 M TRAIN ROWS; FL USES THE FEDERATED 7-CLIENT PARTITION.

Baseline	Test AUC	Δ vs FL natural-by-BS
Last-BLER persistence (`score=ul_bler[t]`)	0.5133	+0.4026
Smoothed-5s persistence (per-group rolling mean)	0.6258	+0.2901
Logistic regression on V3_CONTINUOUS (17 features)	0.6523	+0.2636
Centralized LSTM, 1 epoch (226k steps)	**0.9314**	**−0.0155**
Centralized LSTM, 3 epochs (overfits slightly)	0.9264	−0.0205
LSTM × FedAvg × natural-by-BS (Phase 5, federated)	0.9159	—

0.02–0.03 AUC at flat energy. The paired-bootstrap arch-deltas substantiate this: at IID, $\Delta_{\text{Spiking}-\text{LSTM}}$ AUC ranges from -0.0615 to -0.0739 across the five algorithms (all five cells significant at $p = 0.002$), an order-of-magnitude larger than the algorithm-leverage scale shown in Section VI-C. At Dirichlet $\alpha_{\text{dir}} = 0.05$, Spiking−LSTM widens further to $\Delta = -0.169$ $[-0.186, -0.146]$ for FedAvg. Operator deployment decisions on commodity edge GPU should therefore centre architecture-energy trade-off, with algorithm choice as a secondary tuning lever.

F. Naive baselines: FL machinery vs trivial predictors

Per reviewer feedback (added 2026-05-06), three naive baselines computed on the same test split (1 930 796 sequences, `tr` $\in \{25..27\}$, positive rate 30.57%):

The raw persistence baseline measures the trivial "next second mirrors current second" predictor; per-group lag-1 Pearson correlation on `ul_bler` is -0.0509, confirming BLER is essentially white noise at 1-second granularity. Smoothing over the prior 5 seconds (matching the LSTM input window) raises persistence AUC to 0.6258 but still leaves a +0.29 AUC gap to FL. Logistic regression on the same 17 continuous features the FL models consume reaches 0.6523, indicating linear classification captures partial signal but $\sim$ 26 pp remain for sequence modelling and federation. The smallest gap (FL minus the strongest naive) is +0.26 AUC — substantial enough to justify the FL machinery on this task.

Decomposition (added 2026-05-06): the centralized LSTM run cleanly separates the two factors that the naive-vs-FL gap previously conflated:

- **ML lift** (sequence modelling over linear): centralized LSTM 0.9314 − logistic regression 0.6523 = **+0.28** AUC.
- **Federation cost** (federated vs centralized at equivalent training budget): centralized LSTM 0.9314 − FL natural-by-BS 0.9159 = **+0.015** AUC.

FL "costs" $\sim$ 1.5 pp AUC compared to centralized training on this task — a small price for the privacy / locality / no-pooling benefits FL provides on the O-RAN edge. The FL benefit over LR is not "FL machinery is required to do well", but rather "sequence modelling is required to do well, AND federation is nearly free on top of sequence modelling". The 3-epoch centralized result (0.9264 < 1-epoch 0.9314) suggests centralized training reaches its OOD-test ceiling within 1 epoch on this dataset; further training overfits slightly. FL's 100-round budget (25k gradient steps total $\approx$ 0.1 epochs) is therefore well-matched to the task.

Reproduction: `scripts/baseline_last_bler.py` + `scripts/baseline_logreg.py` + `experiments/run_p1_centralized_lstm.py`; persisted to `artifacts/baselines/{naive,centralized_lstm}_results.json`.

G. Mechanism preview: open question, dataset-structural rather than algorithm-mechanistic

The four findings above raise one mechanistic question: *why does heterogeneity help here when standard FL benchmarks find the opposite?* Section VII develops this in detail. We summarise here so reviewers can navigate the discussion: (a) one candidate that mixed per-client class-imbalance variance with structural specialisation was eliminated after we verified that `fl_v7.py` uses a globally pooled positive-class weight in the BCE loss (the implementation is not client-local), and the global positive rate (30.9%) is balanced rather than sparse; (b) the second candidate "natural-by-BS preserves bs↔slice mixture correlation" was eliminated after measuring per-bs slice mixtures and finding them statistically uniform across all 7 base stations (KL $\approx$ 0); (c) the leading candidate "Dirichlet's per-slice row redistribution destroys structurally-coherent client datasets that natural-by-BS preserves" is supported by two controlled ablations — Section VII-A1's `random_split` (breaks both bs and slice grouping; AUC drops $\sim$ 0.18 below natural-by-BS) and Section VII-A5's per-BS Dirichlet (preserves bs grouping while varying within-bs slice grouping; refutes the strong reading that bs grouping alone suffices). We do not claim a closed-form mechanism in Section I; the empirical finding stands as a deployment-relevant observation.

VII. DISCUSSION

A. Why heterogeneity helps: empirical observation + tested-hypothesis structure

The Section VI-B inversion of the Dirichlet-α_{dir} / AUC monotonicity is the central empirical finding of this paper.

We do *not* present a closed-form mechanism explanation — instead we report the empirical pattern, document a controlled ablation (Section VII-A1) that tests the leading hypothesis, and bound the explanatory candidates with what we measured directly from the dataset.

Setup-level facts (measured, not derived). The training pipeline uses a *globally pooled* positive-class weight in the BCE loss (`fl_v7.py` line 636–637 sums positive counts across all client shards before computing the reweighting), so the BCE loss is identical across clients and any mechanism story relying on per-client class-imbalance variance is incompatible with the code. The empirical positive rate on the train partition is 30.9% (per-slice 35% / 27% / 31%), so this is a balanced-ish binary task, not a sparse-positive-class regime. The dataset's per-bs slice mixture is statistically uniform (KL ≈ 0; fitted Dirichlet $\hat{\alpha_{\text{dir}}} \approx 234\,000$), so natural-by-BS is *not* a low-α_{dir} Dirichlet limit case — the two partitions are orthogonal in their structural axis (`bs_id` grouping vs `slice_id` redistribution).

Candidate explanations for the empirical pattern. Three candidates remain after the above eliminations: (i) per-bs *continuous*-feature distributions differ across base stations even though slice mixtures are uniform — measured per-feature pairwise KL up to 0.08 on `tx_brate_dl_Mbps` and `sum_granted_prbs` (Section VII-A2); (ii) the Dirichlet partition's per-slice row redistribution destroys bs-level grouping that natural-by-BS preserves, depriving each client of a coherent bs-conditioned dataset; (iii) any FedAvg of independently-trained client models on heterogeneous shards may behave better than a FedAvg of clients trained on shards that mix structurally similar rows — a generic heterogeneity-as-implicit-regularisation effect. None of these candidates are unique to FL with imbalanced classification; they are dataset-structure properties that we observed.

1) Controlled ablation: random-split partition: To isolate hypothesis (ii) (bs-grouping preservation), we add a **`random_split`** partition mode that uniformly shuffles all training rows and assigns equal-size shards to 7 clients, breaking *both* `bs_id` and `slice_id` grouping. Per-client sample sizes are within ± 1 row of `len(train)/7`, removing per-client sample-size confounds. Implementation: `partition_clients(mode="random_split", n_clients=7, seed=...)` in `data_v2/partition.py`.

Cells: 3 architectures × FedAvg × `random_split` × 5 seeds = 15, run on 4× Tesla V100-SXM2-32GB matching the Phase 5 training spec. Comparison baseline: the corresponding 30 cells of (3 archs × FedAvg × natural-by-BS × 10 seeds) from Phase 5. Results are summarised in Table III.

`random_split` AUC also collapses to within ± 0.007 of the most-uniform Dirichlet partition ($\alpha_{\text{dir}} = 5.00$) on every architecture — the two partition strategies (Dirichlet $\alpha_{\text{dir}} = 5.00$ over slice; uniformly random over rows) both ignore bs grouping and produce statistically equivalent results. The ~ 0.18 AUC drop relative to natural-by-BS is an order of magnitude larger than the V100-vs-4080 hardware drift bound (~ 0.007, see Section VII-A4 below; ratio $\approx 25\times$). The mechanism direction is therefore consistent with: **partitions that preserve the natural bs grouping outperform partitions that break it (whether by per-slice Dirichlet redistribution or uniform shuffling), and the magnitude of the AUC penalty is consistent across the three sequence backbones.** Two interpretive caveats:

TABLE III
V100 `RANDOM_SPLIT` ABLATION VS PHASE 5 NATURAL-BY-BS BASELINE. TEST AUC MEAN ± STD ON 10 SEEDS (4080, NATURAL-BY-BS) AND 5 SEEDS (V100, `RANDOM_SPLIT`); Δ IS MEAN−MEAN (UNPAIRED ACROSS HARDWARE).

arch	`random_split` (V100, $n = 5$)	natural-by-BS (4080, $n = 10$)	
LSTM	0.7403 ± 0.0027	0.9159 ± 0.0004	–
Mamba	0.7447 ± 0.0077	0.9165 ± 0.0006	–
Spiking-SSM	0.6668 ± 0.0030	0.8529 ± 0.0051	–

- **Δ in Table III is mean−mean (unpaired across hardware), not a paired-bootstrap CI_{95}.** V100 `random_split` was run with 5 seeds; Phase 5 natural-by-BS used 10 seeds; the seed sets do not coincide. A strict paired comparison would require running `random_split` with the same 10 seeds as Phase 5 — we used 5 seeds for V100-time efficiency, and the 25× SNR margin against the Section VII-A4 hardware drift bound makes this protocol gap non-blocking but should be noted.
- **The std comparison in Table III is not strictly fair.** `random_split`'s per-seed std combines two independent variance sources (model-init RNG + partition-shuffle RNG), while natural-by-BS's std reflects only init RNG (the partition is deterministic). Comparing means is valid; comparing stds favours natural-by-BS by construction.

On the Section VI mechanism question: `random_split` breaks both `bs_id` and `slice_id` grouping in one step, so the observed AUC drop is consistent with either candidate (i) "natural-by-BS preserves bs-conditioned signal" or candidate (ii) "Dirichlet's per-slice row redistribution destroys structurally-coherent client datasets" — both are violated by `random_split`. Section VII-A5 below reports a per-BS Dirichlet ablation that preserves bs grouping while varying within-bs slice grouping, disambiguating the two candidates and supporting (ii); Section VII-A2 measures per-bs continuous-feature distributions to bound (i) further. A fully-causal mechanism (e.g., synthetic data with controlled per-bs covariance) is left as open future work.

2) Per-bs continuous-feature distribution differences (Step 2 measurement): We measured pairwise per-bs Kullback–Leibler divergence on every continuous feature (`scripts/step2_mechanism_search.py`, output `artifacts/step2_mechanism_search.json`). The top-5 most-discriminating features by mean pairwise bs-KL are `dl_cqi` (KL = 0.475), `dl_mcs` (0.267), `num_ues` (0.092), `tx_brate_dl_Mbps` (0.081), and `sum_granted_prbs` (0.059). The top two — channel-quality indicator (CQI) and modulation-and-coding scheme (MCS) — are physical-layer features that depend on each gNB's specific RF environment and user-cell distance distribution, and their per-bs distributions differ by an order of magnitude more than the next-most-discriminating features. The remaining

TABLE IV
PER-BS DIRICHLET ABLATION (PHASE 6, `SUB_PER_BS=2`, 5 SEEDS × 3 ARCHS). WITHIN-PHASE-6 Δ AND CI_{95} PAIRED BY SEED VIA 10 000-SAMPLE BOOTSTRAP; NATURAL-BY-BS COLUMN IS UNPAIRED ACROSS n AND HARDWARE.

arch	$\alpha_{\text{dir}} = 0.05$	$\alpha_{\text{dir}} = 10$	Δ (pp)	95% CI (pp)	natural-by-BS
LSTM	0.906	0.831	7.5	[6.6, 8.5]	0.916
Mamba	0.909	0.836	7.2	[6.8, 7.7]	0.917
Spiking	0.816	0.684	13.2	[11.7, 15.1]	0.853

TABLE V
TR-EMBEDDING-BUG CONFOUND ACROSS 3 BACKBONES. "SHRINKAGE" IS $(\Delta\text{gap}_{\text{normal}} - \Delta\text{gap}_{\text{meanfix}})/\Delta\text{gap}_{\text{normal}}$. ALL 3 ARCHS SHOW $\leq 10.2\%$ BUG CONTRIBUTION; RESIDUAL GAPS REMAIN 0.04–0.15 AUC.

arch	gap_normal	gap_meanfix	shrinkage	residual
LSTM	+0.0540	+0.0491	9.2%	0.0491
Mamba	+0.0477	+0.0428	10.2%	0.0428
Spiking-SSM	+0.1580	+0.1545	2.3%	0.1545

continuous features (BLER, RSSI, SINR, buffer occupancy) all have mean pairwise bs-KL below 0.02, confirming that the bs-discriminative signal is concentrated in channel-state and load features rather than in BLER itself. These KL magnitudes are non-zero (vs the per-bs slice-mixture KL ≈ 0 in Step 1) and are consistent with the Section VII-A1 ablation finding: bs grouping carries continuous-feature distributional signal that natural-by-BS preserves but Dirichlet / `random_split` partition spreads across clients. Whether this specific channel-state signal is the *causal* mechanism behind the ~ 0.18 AUC drop, vs an alternative (e.g., bs-conditioned sequence-dynamics learnability), remains an open question the present ablation cannot resolve.

3) Applicability boundary: The inverted-α_{dir} finding is dataset-structural, not algorithm-mechanistic; the conditions under which it should replicate to other near-RT RIC forecasting workloads are detailed in App. A.1.

4) Hardware drift caveat for the random-split ablation: Section VII-A1 cells ran on 4× Tesla V100-SXM2-32GB while Phase 5 ran on RTX 4080. We bound the cross-hardware AUC drift by comparing V100 `random_split` AUC to 4080 Dirichlet $\alpha_{\text{dir}} = 5.00$ AUC (both partition strategies destroy bs grouping): the residual delta is at most 0.0072 (LSTM), 0.0043 (Mamba), 0.0021 (Spiking-SSM), all within seed σ. Hardware drift is therefore bounded above by ~ 0.007 AUC, more than an order of magnitude smaller than the ~ 0.18 mechanism signal (ratio $\approx 25\times$). Full bf16-tensor-core argument and protocol detail in App. A.2.

5) Per-BS Dirichlet ablation: bs grouping is necessary but not sufficient: Section VII-A1's `random_split` ablation could not separate candidate (i) ("natural-by-BS preserves bs-conditioned signal") from (ii) ("Dirichlet's per-slice row redistribution destroys structurally-coherent client datasets") because `random_split` violates both. We test the **strong reading of (i)** — that bs grouping *alone* suffices, regardless of within-bs slice handling — via a per-BS Dirichlet ablation (`per_bs_dirichlet`, `sub_per_bs=2`, $\alpha_{\text{dir}} \in \{0.05, 0.5, 5, 10\}$, 5 seeds × 3 archs = 60 cells, 4× V100): each BS is split into two sub-clients via Dirichlet($[\alpha_{\text{dir}}]$) over `slice_id`; no client sees rows from multiple BSs. At $\alpha_{\text{dir}} = 0.05$ sub-clients are slice-specialists; at $\alpha_{\text{dir}} = 10$ each has near-uniform slice mixture. Headline numbers in Table IV.

Phase 6 (this paper's per-BS Dirichlet ablation, parallel naming to **Phase 5**'s Stage-2 sweep) means over 5 seeds; Δ and CI_{95} are *paired by seed within Phase 6* via a 10 000-sample bootstrap ($n = 5$ caveat: bootstrap percentile CIs at this n are mildly optimistic, but the effect-size-to-bias ratio is $\sim 10\times$ across all three archs). All three within-Phase-6 deltas exclude zero, *and remain non-zero under Bonferroni-corrected $CI_{98.33}$ for the 3-arch family* (LSTM [6.5, 8.6], Mamba [6.7, 7.8], Spiking [11.5, 15.3] pp), so multiplicity correction does not change the verdict. Since bs grouping is invariant across these cells (each shard from exactly 1 BS), AUC degradation can only originate from per-client structural changes. **The strong reading of (i) is therefore inconsistent with these data; (ii) is supported**, consistent with Section VI-B's cross-bs inverted-α_{dir} effect (per-client slice concentration helps in both cross-bs and within-bs Dirichlet partitions).

The residual gap between Phase 6 $\alpha_{\text{dir}} = 0.05$ and the Phase 5 natural-by-BS baseline (LSTM 1.0, Mamba 0.8, Spiking 3.7 pp) is *unpaired across n and hardware* (Phase 5 $n = 10$ on RTX 4080; Phase 6 $n = 5$ on V100; cf. Section VII-A4 hardware-drift bound ≤ 0.007 AUC). It is consistent with two design constraints (`sub_per_bs=2` halves per-client data; per-round sampling fraction drops $5/7 \to 5/14$) but cannot be cleanly decomposed; the within-Phase-6 effect (7–13 pp) dominates this residual.

6) Quantifying the tr-embedding-bug confound: Added 2026-05-06 per reviewer Minor#4. A code-level audit identified that the test traffic-config index `tr` $\in \{25, 26, 27\}$ is encoded via `nn.Embedding(30, 8)`, but the train set only ever indexes rows $\{0..21\}$. Rows 22–29 therefore stay at PyTorch's default $\mathcal{N}(0, 1)$ initialisation and are bit-identical to a fresh seed-0 init at test time (verified across all 3 architectures in `tests/test_audit_invariants.py::test_tr_embedding_rows_22_to_29_byte_identical_to_fresh_`). The reviewer's concern: this random-vector test-time confound might artificially inflate natural-by-BS dominance if the Dirichlet partition relies more on the `tr` feature.

We quantify the impact via inference-only re-evaluation on 54 cells (3 archs × FedAvg × {natural-by-BS, Dirichlet $\alpha_{\text{dir}} = 0.05$} × 9 seeds; `experiments/run_p1_tr_embedding_check.py --archs lstm mamba spiking_expand2`) — load each Phase 5 checkpoint, replace `tr`-embedding rows 22–29 with the mean of trained rows 0–21, re-compute test AUC. Per-architecture summary in Table V.

The bug explains $\leq 10\%$ of natural-by-BS dominance across all 3 backbones; the remaining $\geq 90\%$ is structural, consistent with the Section VII-A1 `random_split` + Section VII-A5 per-BS Dirichlet ablations.

The Spiking-SSM result is particularly informative: it has the largest absolute natural-by-BS gap (0.16 AUC) but the smallest relative bug contribution (2.3%) — i.e., on the ar-

chitecture where C1's effect is strongest, the bug confound is weakest. This rules out the alternative reading "natural-by-BS dominance is mostly a tr-embedding artefact". The reviewer Minor#4 confound is real but immaterial to C1's substantive claim. The bug is a fixable artefact of the embedding-table sizing convention; recommended fix discussed in `artifacts/audit/tr_embedding_audit.md`.

B. Algorithmic flatness implies FedAdam is the ceiling, not the start

Section VI-C establishes that on this dataset the FedAdam-vs-FedAvg gap (Dirichlet-averaged $\approx +0.006$ to $+0.016$ AUC depending on architecture) is the empirical maximum gap any server-side aggregation modification can extract under our (7-client, 71% participation, 100-round) regime. The flatness is *not* a property of FL in general — large-N low-participation FL on image classification regularly shows ≥ 0.05 AUC gaps between FedAvg and FedSAM/FedSWA-class methods. The flatness is a property of *aggregation-noise-dominated* small-N high-participation regimes where the per-round update $(v_t - \theta_{t-1})$ reflects SGD noise across nearly-overlapping client averages rather than coherent drift toward sharp minima.

This explains the negative result on FedDyn-canonical: the canonical Acar 2021 SGD-friendly variant $h_i \leftarrow h_i + (w_l - w_g)$ diverges to NaN under our Adam local optimiser at the 100-round budget, due to a positive-feedback loop between Adam's adaptive scaling and the unbounded h-accumulation. Our Adam-friendly Option-II variant ($h_i \leftarrow h_i - \alpha_{\text{dyn}} \cdot \nabla \mathcal{L}_i(w_t)$ with $\alpha_{\text{dyn}} = 0.01$) reaches centralised parity (test AUC 0.9147 on LSTM IID at 100 rounds, matching the centralised reference). The lesson generalises: server-side aggregation algorithms that perform well in their original SGD regime can require non-trivial recalibration under Adam local optimisers, and the absence of such recalibration is a frequent silent failure mode in the FL benchmark literature.

C. The Mamba × SCAFFOLD × $\alpha_{\text{dir}} = 0.10$ catastrophic interaction

The Section VI-D finding warrants an explicit mechanism account. SCAFFOLD's per-client control variate $c_{\text{local},i}$ is a running estimate of the client's average gradient direction relative to the global average, and the gradient correction $+(c_{\text{global}} - c_{\text{local},i})$ rotates each local update toward an "unbiased" trajectory in expectation. Under high partition skew ($\alpha_{\text{dir}} = 0.10$) the per-client gradient direction is itself highly variable across rounds (each client trains on a small, sparse-positive subset of the train rows), so $c_{\text{local},i}$ is a noisy estimate. The selective-scan SSM in Mamba additionally has an input-dependent dynamics matrix ($\Delta t, B, C$ are all functions of the input — Gu and Dao [33]), making the per-client gradient direction *also* depend on the local partition's input distribution in a way the LSTM's input-independent recurrence does not. The combination — noisy $c_{\text{local},i}$ × input-dependent backbone dynamics — produces a feedback loop where the SCAFFOLD correction overshoots in the direction of the prior-round's noise, and the next round's local training amplifies the overshoot rather than correcting it.

Mamba × FedAvg at the same α_{dir} does not exhibit this failure (std 0.025 at $\alpha_{\text{dir}} = 0.10$ vs 0.083 under SCAFFOLD), nor does LSTM × SCAFFOLD (std 0.036 at $\alpha_{\text{dir}} = 0.10$) where the input-independent recurrence breaks the feedback. The pattern is therefore specific to the *intersection* of input-dependent recurrence and noisy control-variate estimation; it does not generalise to all (state-space-model, control-variate) pairs but should be re-checked whenever SCAFFOLD or its variants are deployed alongside selective state-space architectures (S4, S5, S6, Mamba-2 — the input-dependent SSM family generally) under partition skew. The SCAFFOLD hyperparameter envelope (client-LR, control-variate decay) was not exhaustively swept; see Section VIII for the limitation.

D. Architecture leverage dominates algorithm leverage on commodity edge GPU

The Section VI-E Pareto observation has direct deployment implications. On RTX 4080 (sm_89, TSMC 4N (5 nm class) Lovelace process, 320 W TGP), the $10\times$ energy band between LSTM ($\sim$ 3.5 kJ/cell) and Spiking-SSM ($\sim$ 41 kJ/cell) is dominated by the spiking arm's $T = 5$ simulation timesteps multiplied through the dense post-spike linear layers — the so-called spike-count amplification documented in our prior centralized findings. On a sparsity-aware accelerator (Loihi-2, IBM NorthPole) where post-spike multiplications by zero are skipped, the same architecture would sit closer to the LSTM band; the energy-Pareto disposition between LSTM and Spiking is therefore a property of the deployment hardware, not of the algorithm choice. We discuss the cross-hardware extrapolation in Section VIII.

The within-architecture algorithm spread (≤ 0.03 AUC at flat energy) is small enough that a deployed system should select algorithm based on operational considerations (state size — FedDyn and SCAFFOLD carry per-client h or c variates; communication round count; client straggler tolerance) rather than chase the $+0.01$ AUC gain that FedAdam offers over FedAvg. The Section VI-D catastrophic-interaction caveat is the principal exception: deployments combining selective-SSM backbones with control-variate algorithms should validate on representative high-skew partitions before committing to the combination.

E. Why we do not benchmark FedSWA / FedSCAM

The 2024–2026 sharpness-aware FL family (FedSWA [11], FedSCAM [12], FedMoSWA [52]) is structurally adjacent to FedAdam in that all four algorithms modify the server-side aggregation step. Section II-F develops the mechanism-based argument for why LookAhead-style overshoot is *systematically* dominated by Adam-style adaptive variance damping in our aggregation-noise-dominated regime, and Section VI-C provides the empirical FedAdam ceiling that the LookAhead family cannot exceed without variance-estimation machinery it does not have. The argument is mechanism-level and predictive: in a regime that satisfies our four conditions (small-N, high-participation, short rounds, inverted-α_{dir}), we predict $|\Delta_{\text{FedSWA}-\text{FedAvg}}| < |\Delta_{\text{FedAdam}-\text{FedAvg}}|$. The decision to not benchmark is *not* "we ran out of GPU"; the decision is "the

predicted result does not advance the Section I contributions." We document this explicitly here so a reviewer arguing "you should still benchmark for empirical confirmation" can engage with the predictive claim rather than the decision.

VIII. Limitations and Threats to Validity

We enumerate the limitations and threats to the validity of the Section I contributions. Each item is paired with the contribution(s) it most threatens.

- **L1 (threats contributions 1, 4): Single-hardware energy reporting.** All energy numbers are NVML measurements on a single RTX 4080 (sm_89). Energy on consumer-tier mobile GPUs (RTX 30-series at lower TGP), data-centre accelerators (A100, H100), and sparsity-aware accelerators (Loihi-2) will differ in absolute magnitude and possibly in the architectural ranking. The headline $10\times$ LSTM-vs-Spiking energy ratio is a property of the dense GPU + $T = 5$ spiking-simulation regime; on sparsity-aware hardware the ratio would shrink (prior centralized estimate ≈ 0.49). A cross-GPU robustness sweep (T4, A100) is open future work.
- **L2 (threats contributions 2, 3): 5-algorithm baseline excludes 2024–2026 sharpness-aware FL.** We do not empirically evaluate FedSWA [11], FedSCAM [12], or FedMoSWA [52] on Colosseum/ColO-RAN. The mechanism-based argument in Sections II-F + VI-C + VII-B predicts these methods would perform at or below FedAdam in our regime; empirical confirmation in regimes where the heterogeneity-as-sharpness-wound assumption holds (large-N, low-participation, image classification with $\alpha_{\text{dir}} < 0.1$) is left to future work. Regarding FedBN [13], often suggested as the feature-shift FL baseline: our 3 backbones (LSTM, Mamba, Spiking-SSM) intentionally omit BatchNorm. FedBN's defining server-side rule — skip BatchNorm parameters during aggregation — therefore reduces bit-exactly to FedAvg's complete aggregation on these architectures (proof in `artifacts/audit/fedbn_reduces_to_fedavg.md`). **We additionally ran a 30-cell empirical confirmation** (3 archs $\times$ FedBN $\times$ natural-by-BS $\times$ 10 seeds, 100 rounds, RTX 4080) and verified the reduction on the trained outputs. Per-architecture paired-bootstrap CI_{95} ($n_{\text{boot}} = 10\,000$, matching the Section IV-E protocol) on the (FedBN $-$ Phase-5-FedAvg) per-seed test-AUC delta: LSTM $\Delta = 0$ $[0, 0]$ (bit-exact 10/10 seeds); Mamba $\Delta = +1.0\text{e}{-6}$ $[-4\text{e}{-6}, +8\text{e}{-6}]$; Spiking $\Delta = +4.9\text{e}{-4}$ $[-4.4\text{e}{-3}, +4.3\text{e}{-3}]$. **All 3 architectures CI_{95} include zero** — empirically equivalent to FedAvg. The Spiking CI width ($\sim 9\text{e}{-3}$) is wider than the inter-algorithm gaps reported in Section VI-C (FedAdam-vs-FedAvg ≈ 0.006–0.016 AUC), but this BF16-numerical-noise floor affects all algorithm-pair comparisons equally and is captured in Section VI-C's paired-bootstrap CIs by the same mechanism. A 3-cell diagnostic further isolates the Spiking-arch $\sim 3.5\text{e}{-3}$ cross-time delta as environmental drift over the 6 weeks between Phase 5 and the FedBN sweep (likely CUDA driver / PyTorch / matmul-order changes; BF16 numerics are not bit-reproducible across such updates), not algorithmic difference. FedAvg is already in our 5-algorithm baseline. The reported FedBN benefit on cellular feature-skew tasks (arXiv:2508.08479 [8] reports $+0.01$–0.03 AUC over FedAvg) is BN-specific and structurally not transferable to no-BN backbones. MOON [50] is structurally deferred (preregistered) because its `forecaster_encode_fn` does not generalise cleanly to selective-SSM or spiking-SSM activations.
- **L3 (threats contribution 1): Round count is short by image-classification standards.** We use 100 communication rounds $\times$ 50 max-steps per round $\times$ 5 clients sampled per round = **25 000 total gradient-steps per cell** (the figure that matches the audit-corrected centralised budget from our prior centralized work). Per individual client, each client is sampled with probability $5/7$ per round and executes 50 steps when sampled, so the *expected* per-client gradient-step count is $100\times(5/7)\times 50 \approx 3\,571$ steps per client per cell. Both numbers are short relative to the 1 000-round runs typical of FedSWA / FedSCAM evaluations. The predicted-flatness argument holds at this round count; whether it relaxes at longer round counts is open. A 1 000-round LSTM $\times$ FedAvg $\times$ IID extrapolation experiment is cheap ($\sim$ 3 hr GPU on RTX 4080) and is a candidate future ablation.
- **L4 (threats contribution 1): 7-client natural-deployment scale.** All experiments use 7 clients — the natural base-station count of the public ColO-RAN release, not a researcher choice — at $5/7 = 71\%$ participation per round. Statements about FL behaviour at the cross-device 1 000-client / 10% participation regime [51] do not generalise from this study. The Section VII-B argument explicitly acknowledges this: contribution 2's flatness is a property of high-participation FL at small-N (here, the dataset-imposed $N = 7$), not of FL in general. Larger-scale ColO-RAN releases or alternative O-RAN testbeds (e.g., OpenAirInterface 5G, srsRAN) could let future work probe the cross-device limit, subject to non-trivial engineering effort to align telemetry format with our preprocessing pipeline (OAI 5G's logging differs substantially; srsRAN lacks integrated FL training infrastructure).
- **L5 (threats contribution 5): Reproducibility infrastructure not yet released.** The full 4-axis table maps deployment configurations to AUC / F1 / energy across 90 groups; the contribution 5 reproducibility claim depends on the Section V release of the spec-driven YAML pipeline + paired-bootstrap statistics + Croissant metadata + Zenodo DOI under Apache-2.0. At submission time the artefact is mature in `src/fl_oran/`, `experiments/specs/`, and `tests/` but the Section V reproducibility section is staged for the camera-ready release; we will provide a Dockerfile + `requirements.lock` + demo notebook in the supplementary archive.
- **L6 (threats contribution 4): Pareto Spiking arm assumes hardware sparsity unavailable on RTX 4080.** The Section VI-E architecture-leverage claim treats Spiking as a distinct vertical band at $\sim$ 41 kJ/cell. On the

deployed sparsity-aware target the Spiking arm's energy would compress (per Section VII-D and prior centralized analysis), and the Pareto envelope between LSTM and Spiking would tighten substantively. Operators targeting commodity GPU should read the Section VI-E ranking as-is; operators targeting Loihi-2 / NorthPole should read the ranking with a 2–3× downward correction on Spiking energy.

- **L7 (threats contribution 1, Section VII-A): Mechanism for the inverted-α_{dir} finding is partially open.** Section VII-A reports a `random_split` controlled ablation (15 cells × 3 archs × 5 seeds, run on 4× V100-SXM2-32GB) that tests the leading hypothesis (natural-by-BS preserves bs-conditioned continuous-feature signal). Two prior candidates (per-client class-imbalance specialisation; bs↔slice mixture correlation) were eliminated by direct measurement before the ablation (Sections VII-A + VII-A2). If the `random_split` ablation also fails to recover the Section VI-B monotonicity, the Section VI mechanism question is left open and is the highest-priority follow-up for the camera-ready revision.
- **L8 (threats contribution 2): SCAFFOLD interaction not exhaustively swept.** The Section VI-D catastrophic-interaction finding uses canonical SCAFFOLD hyperparameters. A SCAFFOLD client-LR / control-variate-EMA grid sweep on Mamba × $\alpha_{\text{dir}} = 0.10$ would establish whether the failure is recoverable in some hyperparameter region; we report the interaction with canonical hyperparameters to establish the existence of the failure mode rather than its precise envelope.
- **L9 (threats Sections III, VI, VII-A): `pos_weight_split=train` is one of three valid choices.** The BCE loss reweighting is globally pooled across all clients (Section VII-A), so the *direction* of all findings is invariant to this choice; full alternative-computation discussion in App. C.1.
- **L10 (threats contribution 5 reproducibility): bf16 mixed-precision training versus fp32.** All Phase 5 cells use bf16; we do not report an fp32 verification cell. Full numerical-precision argument in App. C.2.
- **L11 (threats contribution 1, Section VII-A): Per-client compute budget split as 100 rounds × 50 max-steps versus alternative splits.** We treat round count as a fixed reproducibility anchor; whether the Section VI-C FedAdam-saturates-headroom finding survives at alternative splits is open. Full discussion in App. C.3.
- **L12 (threats contribution 1): SLA-threshold sensitivity not exhaustively swept.** The 10% BLER threshold is the canonical ColO-RAN SLA gate [1]; a $\{5\%, 15\%, 20\%\}$ sweep is the highest-leverage open ablation after Section VII-A1. Full discussion in App. C.4.
- **L13 (threats contribution 5 reproducibility): Bootstrap CI uses percentile interval, not BCa.** At our $n = 10$ paired seeds with approximately symmetric per-seed delta distributions, the BCa bias-correction term is bounded by $O(1/\sqrt{n}) \approx 0.32$ standard errors — smaller than seed σ on every reported finding except Section VI-D's Mamba×SCAFFOLD. Full justification in App. C.5.
- **L14 (threats contribution 2): FedAdam hyperparameter sensitivity untested.** $\beta_1 = 0.9$, $\beta_2 = 0.99$, $\eta_{\text{server}} = 0.01$ follow Reddi et al. 2021 [47] preregistered values; $(\beta_1, \beta_2, \eta_{\text{server}})$ sensitivity sweep deferred. Full discussion in App. C.6.

IX. Conclusion

We have presented the first comprehensive 4-axis federated-learning benchmark on the publicly-available Colosseum/ColO-RAN testbed for slice-level SLA-violation prediction, sweeping {LSTM, Mamba, Spiking-SSM} × {FedAvg, FedProx, FedAdam, SCAFFOLD, FedDyn} × {natural-by-BS, Dirichlet $\alpha_{\text{dir}} \in [0.05, 5.0]$} × 10 seeds = 900 cells with NVML idle-baseline-subtracted training-energy measurement on commodity-tier RTX 4080, augmented by a 15-cell V100 ablation that tests the leading mechanism hypothesis for the headline inverted-α_{dir} finding.

The principal finding is that **preserving the natural base-station client grouping outperforms every parametric Dirichlet α_{dir} partition across all 3 architectures × 5 algorithms in our setup**: AUC monotonically increases as $\alpha_{\text{dir}} \to 0$ within the Dirichlet sweep, but never matches the natural-by-BS baseline. We do not claim a closed-form mechanism for this inversion in the conclusion; Section VII-A reports a controlled `random_split` ablation that tests the leading "natural-by-BS preserves bs-conditioned signal" hypothesis, and two prior mechanism candidates (per-client class-imbalance specialisation; bs↔slice mixture correlation) were eliminated by direct measurement. The empirical inversion is the deployment-relevant claim; the mechanism is partially open. The three secondary findings (FedAdam saturating algorithmic headroom; Mamba × SCAFFOLD catastrophic interaction; architecture leverage dominating algorithm leverage on energy) are detailed in Section I contributions 2–4 and Sections VI-C–VI-E; we do not restate the numbers here.

The community-level implication is the one operationalisable insight beyond the per-cell numbers: standard FL practice — *partition clients by inducing artificial uniformity, then deploy sharpness-aware aggregation to suppress the heterogeneity-induced noise* — is the wrong protocol for O-RAN edge-FL when natural client structure (one gNB, one client) is already heterogeneous in the prediction-relevant axis. The recommendation, in our setup, reverses: **preserve the natural client partition, and require synthetic re-partitioning to be justified by a specific operational constraint** (e.g., privacy-aware aggregation) rather than adopted as a default benchmarking convention. For O-RAN edge-FL practitioners, the operational recommendation is concrete: deploy on the natural base-station partition rather than reshuffle clients to enforce IID-like uniformity; select architecture for the energy-AUC trade-off (LSTM at the low-energy frontier on commodity GPU; Spiking-SSM remains a research direction pending sparsity-aware accelerator availability — its 10× higher energy on dense GPU is hardware-specific, not a fundamental architectural property, and a deployment recommendation must wait until target neuromorphic / sparsity-aware hardware is empirically characterised, see Section VII-D

+ Section VIII L1, L6); choose algorithm based on operational constraints (state size, communication overhead, straggler tolerance) rather than chase ≤ 0.02 AUC gains, with the explicit caveat that selective-SSM $\times$ control-variate combinations need partition-aware validation before deployment.

Upon paper acceptance, we will release the full benchmark artefact — spec-driven YAML pipeline, pre-flight algorithm validation, paired-bootstrap statistical inference, NVML training-energy instrumentation, Croissant dataset metadata, and a Zenodo-archived release tag — as a reference baseline for further FL-on-O-RAN research.

A. Future work

Five open directions, each detailed in App. D:

- **Mechanism isolation re-run** — `sub_per_bs=2` with `clients-per-round=10` to disentangle slice-mixing from per-round participation in the Section VII-A5 residual gap [53], [54].
- **BLER-threshold sensitivity** — perturb the 10% gate to $\{5, 15, 20\}\%$ to confirm operational robustness [55].
- **Scaling beyond** $N = 7$ — test whether inverted-α_{dir} holds at the 100-client regime [56], [57].
- **Variance-aware sharpness** — FedSCAM [12] and FedSynSAM [58] as candidates beyond the FedAdam ceiling (Section VII-B).
- **Privacy + neuromorphic** — over-the-air DP [59] and Loihi 2 sparse-RNN deployment [60], [61].

REFERENCES

[1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "ColO-RAN: Developing machine learning-based xApps for Open RAN closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, 2022.

[2] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečný, H. B. McMahan, V. Smith, and A. Talwalkar, "LEAF: A benchmark for federated settings," *arXiv:1812.01097*, 2018.

[3] F. Lai, Y. Dai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "FedScale: Benchmarking model and system performance of federated learning at scale," in *ICML*, 2022.

[4] o. Song, "FLAIR: Federated learning annotated image repository," 2022.

[5] F. Granqvist *et al.*, "pfl-research: Simulation framework for accelerating federated learning research," in *NeurIPS Datasets and Benchmarks*, 2024, arXiv:2404.06430.

[6] o. Shen, "STEP: A unified spiking transformer evaluation platform for fair and reproducible benchmarking," *arXiv:2505.11151*, 2025.

[7] (authors), "Fed-LSTM: LSTM forecaster for slice-level traffic in cellular FL," 2022.

[8] ——, "FedAvg/FedProx/FedBN benchmark on five throughput-prediction datasets," 2025, arXiv:2508.08479.

[9] Mangi *et al.*, "When Clients Drift: Regime-aware aggregation in 6G RAN telemetry," 2026.

[10] L. Bonati *et al.*, "Colosseum: Large-scale wireless experimentation through hardware-in-the-loop network emulation," *IEEE Open Journal of the Communications Society*, Aug. 2024.

[11] J. Liu *et al.*, "FedSWA: Federated stochastic weight averaging," in *ICML*, 2025, arXiv:2507.20016.

[12] "FedSCAM: Sharpness-aware clustering for federated learning," 2026, arXiv:2601.00853 (Jan 2026).

[13] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on non-IID features via local batch normalization," in *ICLR*, 2021, arXiv:2102.07623.

[14] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," 2020, arXiv:2007.14390.

[15] O. Shchur *et al.*, "fev-bench: A realistic benchmark for time series forecasting," 2025, arXiv:2509.26468.

[16] (authors), "Statistical federated learning for beyond-5g SLA-constrained RAN slicing," *IEEE*, 2021, source markdown lists no arXiv ID; specific paper requires external lookup at camera-ready.

[17] ——, "CDF-aware federated learning for RAN slicing under long-term SLA constraints," *IEEE*, 2021, source markdown lists no arXiv ID; specific paper requires external lookup at camera-ready.

[18] J. Groen, Z. Yang, D. Muruganandham, M. Belgiovine, L. Ying, and K. Chowdhury, "From classification to optimization: Slicing and resource management with TRACTOR," 2023, arXiv:2312.07896.

[19] R. Barker, A. Ebrahimi Dorcheh, T. Seyfi, and F. Afghah, "REAL: Reinforcement learning-enabled xApps for experimental closed-loop optimization in O-RAN with OSC RIC and srsRAN," in *IEEE ICC Workshops*, 2025, arXiv:2502.00715.

[20] Hayek *et al.*, "Empirical evaluation of federated learning convergence over heterogeneous COTS networks," 2025, arXiv:2504.04678.

[21] J. Chen, Y. Yang, S. Deng, D. Teng, and L. Pan, "SpikMamba: When SNN meets Mamba in event-based human action recognition," in *6th ACM International Conference on Multimedia in Asia (MMAsia)*, 2024, arXiv:2410.16746.

[22] J. Qin and F. Liu, "Mamba-Spike: Enhancing the Mamba architecture with a spiking front-end for efficient temporal data processing," in *Computer Graphics International (CGI)*, 2024, arXiv:2408.11823.

[23] (authors), "Spiking point Mamba for 3D point cloud processing," in *ICCV*, 2025, author list TBD at camera-ready.

[24] ——, "SpikingSSMs: Sparse-parallel spiking state-space models," in *AAAI*, 2025, author list TBD at camera-ready.

[25] Y. Huang, J. Tang, C. Wang, Z. Wang, J. Zhang, Z. Lu, B. Cheng, and L. Leng, "SpikingMamba: Towards energy-efficient large language models via knowledge distillation from Mamba," *Transactions on Machine Learning Research (TMLR)*, 2026, arXiv:2510.04595.

[26] Y. Pan, Y. Feng, J. Zhuang, S. Ding, H. Xu, Z. Liu, B. Sun, Y. Chou, X. Qiu, A. Deng, A. Hu, S. Wang, P. Zhou, M. Yao, J. Wu, J. Yang, G. Sun, B. Xu, and G. Li, "SpikingBrain: Spiking brain-inspired large models," 2025, arXiv:2509.05276.

[27] Y. Venkatesha, Y. Kim, L. Tassiulas, and P. Panda, "Federated learning with spiking neural networks," *IEEE Transactions on Signal Processing*, 2021, arXiv:2106.06579.

[28] (authors), "FedLEC: Federated learning with spiking neural networks under label-skew non-iid," in *IJCAI*, 2025, arXiv:2412.17305.

[29] M. Abbasihafshejani, A. Maiti, and M. Jadliwala, "Spiking neural networks in vertical federated learning: Performance trade-offs," 2024, arXiv:2407.17672.

[30] M. V. Nguyen, L. Zhao, B. Deng, and S. Wu, "The robustness of spiking neural networks in federated learning with compression against non-omniscient Byzantine attacks," 2025, arXiv:2501.03306.

[31] D. Aksu, J. Martinez del Rincon, and I. Alouani, "Privacy in federated learning with spiking neural networks," 2025, arXiv:2511.21181.

[32] (authors), "Firing-rate sensitivity to differential privacy in federated spiking neural networks," in *ICASSP*, 2026, arXiv:2602.12009 (Feb 2026).

[33] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *COLM*, 2024, arXiv:2312.00752.

[34] Shen *et al.*, "Is conventional SNN really efficient? a perspective from network quantization," in *CVPR*, 2024, arXiv:2311.10802; venue: CVPR 2024.

[35] A. Asperti, D. Evangelista, and M. Marzolla, "Dissecting FLOPs along input dimensions for GreenAI cost estimations," in *LOD*, 2021, arXiv:2107.11949.

[36] J. Yik *et al.*, "NeuroBench: A framework for benchmarking neuromorphic computing algorithms and systems," *Nature Communications*, 2025.

[37] J.-W. Chung *et al.*, "The ML.ENERGY benchmark: Measuring inference energy of generative AI models," 2025, arXiv:2505.06371.

[38] ——, "Where do the joules go? diagnosing inference energy consumption," 2026, arXiv:2601.22076.

[39] P. Spyra, "Beyond backpropagation: Exploring innovative algorithms for energy-efficient deep neural network training," 2025, arXiv:2509.19063.

[40] Q. Li, Y. Diao, Q. Chen, and B. He, "NIID-Bench: Federated learning on non-IID data silos: An experimental study," in *ICDE*, 2022, arXiv:2102.02079.

[41] C. J. Mclaughlin and L. Su, "Personalized federated learning via feature distribution adaptation," in *NeurIPS*, 2024, arXiv:2411.00329.

[42] Borazjani *et al.*, "Embedding-based heterogeneity definitions for federated learning beyond label-skew," 2025, arXiv:2503.14553.

[43] D. Caldarola, B. Caputo, and M. Ciccone, "Improving generalization in federated learning by seeking flat minima (FedSAM)," in *ECCV*, 2022.

[44] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," 2019, arXiv:1901.09948; canonical surrogate-gradient reference underlying snntorch's `ATan` surrogate. The "Wei et al. 2018" attribution in source markdown corresponds to no verifiable paper; this Neftci 2019 survey is the authoritative reference for the surrogate- gradient method snntorch implements. Cite key retained to avoid main.tex churn.
[45] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.
[46] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks (FedProx)," in *MLSys*, 2020.
[47] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, "Adaptive federated optimization (FedAdam)," in *ICLR*, 2021.
[48] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *ICML*, 2020.
[49] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization (FedDyn)," in *ICLR*, 2021.
[50] Q. Li, B. He, and D. Song, "Model-contrastive federated learning (MOON)," in *CVPR*, 2021.
[51] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," in *NeurIPS Workshop on Federated Learning*, 2019, arXiv:1909.06335.
[52] J. Liu *et al.*, "Improving generalization in federated learning with highly heterogeneous data via momentum-based stochastic controlled weight averaging (FedMoSWA)," in *ICML*, 2025, arXiv:2507.20016 (companion algorithm to Liu2025_FedSWA; same paper, separate cite key for the momentum variant).
[53] Borges *et al.*, "Per-axis isolation methodology for FL heterogeneity studies," 2025, arXiv:2503.17070.
[54] (authors), "ProFed: Proximity-partition framing for FL," 2025, arXiv:2503.20618.
[55] ——, "Rare-event metric stability under threshold perturbation," 2025, arXiv:2504.16185.
[56] Chiarani *et al.*, "FL at the 100-client regime," 2025, arXiv:2503.12435.
[57] Sezgin *et al.*, "Cross-device FL scaling study," 2025, arXiv:2509.11421.
[58] (authors), "FedSynSAM: Trajectory-matched sharpness-aware federated learning," 2026, arXiv:2602.11584.
[59] ——, "Differential privacy via over-the-air channel noise in federated learning," 2026, arXiv:2510.23463.
[60] ——, "Neuromorphic FL deployment on Loihi 2 with sparse RNN kernels," 2025, arXiv:2502.01330.
[61] ——, "Sparse RNN kernels for neuromorphic inference," 2026, arXiv:2604.27004.